Interpretable Multi-Class Abnormality Classification from Volumetric Chest CT using Dictionary Learning

An undergraduate project report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for the

Degree of the Bachelor of the Science of Engineering Honours

by

K.R.C.S. Kekulawala - EG/2021/4609
B.G.P.L. Thejana - EG/2021/4828
B.G.P.U. Thilakshana - EG/2021/4831
M.D.Y.B. Kularathne - EG/2021/4620

....
Dr. Kaveen Liyanage (Supervisor)
Ms. Sithara Mahagama (Co-Supervisor)

Abstract

Computer Aided Diagnosis (CAD) from volumetric chest Computed Tomography (CT) requires clinically interpretable methods to support radiological decision-making. Current State-of-the-Art (SotA) deep learning models often rely on post-hoc explainability methods such as Grad-CAM, that provide approximate localizations and lack direct traceability to the model's decisions. Furthermore, SotA models rarely support simultaneous multi-class classification and segmentation, leading to a fragmented approach to diagnosis. This also hinders the ability to benchmark CAD approaches across multiple dimensions of performance. This study aims to address these challenges through two contributions: (i) We implement a unified CAD framework that uses inherently interpretable dictionary learning methods, where detected abnormalities can be represented as a linear combination of "atoms", allowing predictions to be projected back to relevant anatomical structures in the CT volume. (ii) We develop an open-source evaluation framework to benchmark SotA models across three axes: classification, segmentation and explainability. We use a subset of the ReXGroundingCT dataset which provides voxel-level 3D segmentation masks corresponding to volumetric scans in the CT-RATE dataset. Based on expert radiologist recommendation and dataset availability, pre-processed volumes are patch-sampled for normal lung parenchyma and two selected pulmonary abnormalities. Using the proposed benchmarking framework, we evaluate four dictionary learning approaches against five SotA models, where our approach achieved 85% classification F1 score and 37% global hit-rate for segmentation, while providing clinically interpretable voxel-level localizations without the need for post-hoc explanations.

Acknowledgments

We would like to express our sincere gratitude to Dr. Kaveen Liyanage (Supervisor), Mrs. Sithara Mahagama (CoSupervisor), and Dr. Subhodini Indrapala (Consultant Radiologist at Teaching Hospital, Karapitiya), for their invaluable guidance, support, and constructive feedback throughout the course of this project. Their expertise, mentorship, and encouragement have been immensely helpful in shaping the direction and quality of this research. We are grateful to the Department of Electrical and Information Engineering for providing access to high-performance computing resources and research facilities that were essential for conducting this project. The department's commitment to fostering an environment conducive to research and innovation is greatly appreciated. We would like to acknowledge the creators and maintainers of the publicly available datasets used in this research, particularly the CT-RATE and RexGroundingCT datasets and other medical imaging resources that have made this work possible. These datasets have been crucial for developing and validating the proposed methods. We extend our thanks to the developers and maintainers of the open-source frameworks and libraries used in this project, which significantly contributed in the implementation of this research.

Preface

This project report is submitted to the Department of Electrical and Information Engineering, Faculty of Engineering, University of Ruhuna, Sri Lanka, in partial fulfilment of the requirements for the Degree of Bachelor of the Science of Engineering Honours. The work described here represents a collaborative project carried out by the authors under the supervision of Dr. Kaveen Liyanage and the co-supervision of Ms. Sithara Mahagama.

This report presents the research and development carried out as part of our final year group project, Interpretable Multi-Class Abnormality Classification from Volumetric Chest CT using Dictionary Learning. The project addresses two limitations of current deep learning based approaches to automated abnormality detection: their reliance on post-hoc explainability techniques that provide only approximate, loosely traceable localizations, and the absence of a unified way to benchmark models across classification, segmentation, and explainability simultaneously. To this end, the project makes two contributions. First, we propose an inherently interpretable dictionary learning framework that learns compact volumetric atoms representing normal lung structures and disease-related radiological patterns, allowing detected abnormalities to be back-projected to the original voxel space and traced directly to the anatomical structures that drove the diagnostic decision. Second, we develop a unified evaluation framework that benchmarks state-of-the-art models on a publicly available 3D chest CT dataset across three axes: classification accuracy, segmentation quality, and explainability.

The report covers the theoretical background, the proposed dictionary learning methods, its implementation, and the design of the evaluation framework, followed by an analysis of the experimental results. Our method is benchmarked against state-of-the-art deep learning baselines using this framework, and the results show that it achieves competitive classification and segmentation performance while offering improved interpretability.

Contents

List of Figures

List of Tables

Acronyms

CXR	-	Chest X-Ray
CT	-	Computed Tomography
CAD	-	Computer-Aided Diagnosis
CNN	-	Convolutional Neural Network
ViT	-	Vision Transformer
XAI	-	Explainable Artificial Intelligence
GGO	-	Ground-Glass Opacity
GGN	-	Ground-Glass Nodule
SotA	-	State of the Art
PACS	-	Picture Archiving and Communication Systems
LIME	-	Local Interpretable Model-Agnostic Explanations
SHAP	-	SHapley Additive exPlanations
DeepLIFT	-	Deep Learning Important FeaTures
CAM	-	Class Activation Mapping
Grad-CAM	-	Gradient-weighted CAM
HU	-	Hounsfield Unit

Chapter 1

Introduction

Developing a unified pipeline for multi-class abnormality classification and grounded localization addresses two of the most critical imperatives in clinical AI adoption: diagnostic accuracy and explainability. Specifically, utilizing dense pixel-level segmentation as the mechanism for localization forces the architecture to provide voxel-level visual evidence for its diagnostic claims. This ensures that the model is accountable and its claims are supported by clinical evidence.

1.1 Background

Lung disease remains a leading cause of global morbidity and mortality [1]. Research shows that early detection of pulmonary diseases significantly improves patient outcomes. For this, radiological imaging tools such as Chest X-Ray (CXR) and Computed Tomography (CT), plays an important role. However, the increasing volume of such imaging data increases radiologists' workload, leading to delays and potential errors in diagnosis. This has motivated the development of automated Computer-Aided Diagnosis (CAD) systems, which use computational algorithms and Artificial Intelligence (AI) to assist in tasks such as abnormality detection, classification, and segmentation [2].

Our research mainly focuses on the development of a CAD system using 3 Dimensional (3D) CT data. While CXR is commonly used as the first imaging test due to its low cost and low radiation exposure, studies show that their sensitivity to abnormailities is limited [3]. Comparatively, CT scans produce high resolution cross-sectional slices that helps detect smaller, early-stage, and subtle lung pathologies that are frequently missed by standard CXR.

However, CT analysis is complex and time-consuming, requiring radiologists to review hundreds of slices per scan, which are often inconsistent due to artifacts, anatomical variations and different scanners/protocols. Furthermore, different anatomical structures can share identical grayscale values. Telling them apart requires expert clinical judgment which is often subjective. High sensitivity can lead to detection of clinically insignificant abnormalities, while visual fatigue can lead to missed findings [4].

To address these challenges, recent research and clinical deployments of AI-based CAD systems are fundamentally changing how radiologists approach CT analysis. AI models such as 3D Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) have shown to reduce interpretation time, and improve lesion detection sensitivity compared to traditional manual methods. Many of these high perform-

ing models are Deep Learning based, which are criticized for their black box nature, where their internal reasoning is difficult to interpret. This lack of interpretability is a significant barrier for clinical adoption, as radiologists require evidence to support a model's diagnostic claims.

This has motivated the development of explainable AI (XAI) methods such as Gradient-weighted Class Activation Mapping (Grad-CAM) that helps to visualize exactly which parts of an image a model focuses on when making a specific prediction. Such XAI methods are highly valuable for building clinical trust in deep learning models. However, they often lack the fine-grained, pixel-level precision required to identify tiny lesions, sometimes highlighting incorrect or confounding features [5].

Recent advances in sparse representation models such as Sparse Autoencoders and Dictionary Learning have shown to provide inherent interpretability in medical imaging. This stems from their ability to reconstruct complex, high-dimensional data as a linear combination of "atoms" or features. This allows the use of efficient algorithms to easily map the output sparse domain back to the original input feature space, providing a linear relationship between the model's decision and anatomical structures. Our work builds on these advances by applying dictionary learning to 3D Chest CT data for multi-class abnormality classification and grounded localization.

1.2 Problem Statement

The clinical utility of Deep Learning based approaches is hindered by a lack of interpretability. While tools like Grad-CAM provide post-hoc explanations to model decisions, these are often insufficient for medical professionals who require a granular understanding of the model's decision making process. In high stakes environments, a model that produces a diagnosis without transparency in its reasoning is difficult to integrate into clinical workflows, as it lacks the accountability necessary for legal and ethical compliance.

1.3 Objectives and Scope

The primary goal of this study is to address the limitations of existing AI-based approaches for abnormality classification and localization in chest CTs, by developing an inherently interpretable dictionary learning framework. This approach aims to enable transparent, anatomically grounded decision making, where each classification outcome can be traced back to specific pathological patterns. This aligns with clinical reasoning and facilitates validation and adoption in radiological workflows. The specific objectives are as follows:

1.3.1 Objectives

- Implement a Dictionary Learning based CAD system using a publicly available annotated 3D chest CT dataset.
- Map dictionary atoms back to anatomical features and visualize their spatial contribution maps.

- Develop a unified evaluation framework for assessing the performance and interpretability of the proposed approach.
- Benchmark model performance against SotA classification and segmentation models on the same dataset.

1.3.2 Scope

The scope of this study focuses on the development and evaluation of a dictionary learning based approach for classification and grounded localization of lung abnormalities using volumetric 3D CT data, with the following limitations:

- This study is limited to 2 thoracic abnormalities: Pulmonary Nodules, and Ground-Glass Opacities, based on expert radiologist recommendation and dataset availbility.
- This study will utilize a single publicly available dataset for training and evaluation, which may limit generalizability to other populations and scanner types.
- The proposed approach will be limited only to interpretable methods, and will not explore approaches that provide better performance for less explainability.

Chapter 2

Literature Review

2.1 Clinical Background

Chronic respiratory diseases account for more than 4 million deaths globally per year [1]. Research shows that early detection of pulmonary diseases, particularly lung cancer and Chronic Obstructive Pulmonary Disease (COPD) dramatically lowers mortality rates. Two landmark studies, the National Lung Screening Trial (NLST) and the Nederlands-Leuvens Longkanker Screenings Onderzoek (NELSON) trial, have shown that low-dose CT (LDCT) screening reduces lung cancer mortality by 20% to 33% compared to CXR [6, 7].

2.1.1 Computed Tomography (CT)

CTs generate 3D structural information by acquiring 2D radiographic projections of an object from many angles and computationally reconstructing them into a stack of cross-sectional slices via algorithms such as filtered backprojection [8]. Figure 2.1 shows the Backprojection reconstruction process for a single CT slice.

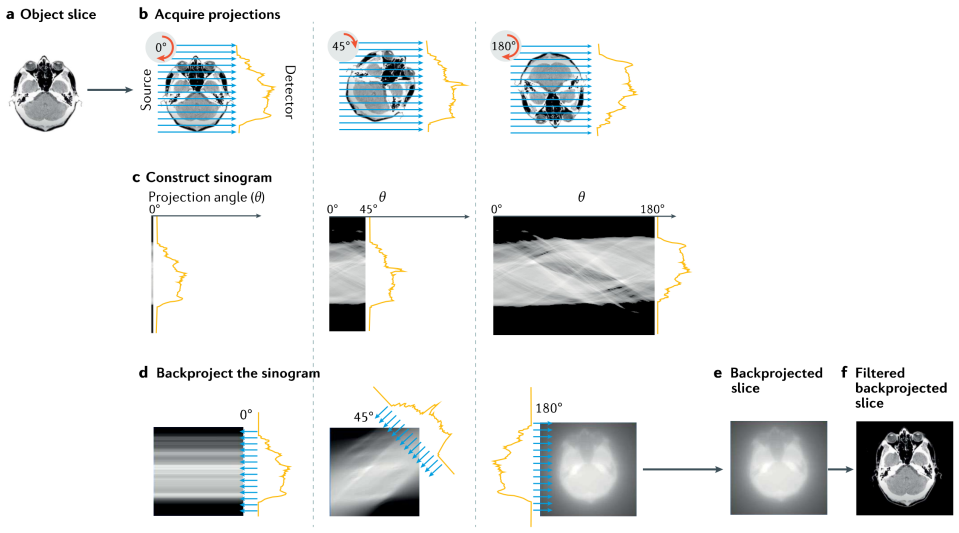

Figure 2.1: Backprojection reconstruction method for a single slice. (a - Original object slice. b - Set of projections collected at different angles. c - Sinogram resulting from many projections. d - Process of backprojecting the sinogram. e - Final backprojected imaqe. f - Equivalent filtered backprojection image).

Compared with other imaging modalities, CT's central advantage is its non-destructive, quantitative access to internal 3D structure. CT can visualize and quantify internal features throughout an intact volume, enabling longitudinal or repeated imaging of the same specimen over time. Attenuation-contrast CT, the dominant mode in clinical practice, works well for high-atomic-number, strongly attenuating structures such as bone or calcified tissue and, with contrast agents, vascular or soft-tissue structures. However, it provides relatively poor contrast between materials of similar composition and density (e.g., different soft tissues), a limitation that phase-contrast and other advanced modalities partially address, though these are less widely deployed clinically due to hardware and dose constraints.

Interpreting 3D chest CT specifically is complicated by several factors described in the imaging literature. Reconstructions are inherently affected by noise and artifacts, motion from breathing or cardiac movement, ring artifacts, and the partial volume effect, where a single voxel spans more than one tissue type, producing intermediate intensities that blur boundaries between structures. These effects broaden and overlap the intensity histograms of different tissues, making simple threshold-based segmentation unreliable and often necessitating boundary-based, region-growing, or learning-based segmentation methods instead. These challenges motivate the shift toward automated, learning-based segmentation approaches capable of handling such structural diversity without extensive manual tuning per dataset.

2.1.2 Pulmonary Abnormalities

Our study focuses on the detection and segmentation of five common pulmonary abnormalities:

1. Ground-Glass Opacity (GGO)
2. Pulmonary nodules
3. Atelectasis
4. Consolidation
5. Bronchiectasis

of which the first two were taken as the primary targets for the initial phase of this study.

A nodule is a discrete, roughly rounded area of increased opacity within the lung parenchyma, generally classified as either a solid nodule or a ground-glass nodule (GGN) based on its internal composition. A GGO refers specifically to a hazy area of increased lung attenuation on CT that does not obscure the underlying bronchial or vascular structures passing through it. GGNs are further subdivided into pure GGNs, which contain no solid component, and mixed (subsolid) GGNs, which contain a solid portion alongside the ground-glass components.

These findings are clinically important because GGNs are frequently associated with early lung cancer and its preinvasive precursors. Notably, GGNs are not uniformly benign or malignant risk indicators equivalent to solid nodules. GGNs are, in fact, more likely to be malignant than solid nodules, and the malignant rate of mixed GGNs is higher than that of pure GGNs.

Early detection of these abnormalities is essential because lung cancer prognosis is highly stage-dependent, and most patients are still diagnosed too late for optimal outcomes. Despite improvements in treatment, the overall five-year survival rate for lung cancer remains only about 10-20% in most countries, largely because most patients are diagnosed at a late stage.

Despite their importance, GGOs and pulmonary nodules remain difficult to detect and characterize reliably. Nodules may be small relative to other anatomical structures in a chest CT, and GGO may be low-contrast against surrounding tissue. These challenges make manual interpretation difficult and motivate the shift toward automated, learning-based segmentation approaches.

2.2 Computer-Aided Diagnosis (CAD) Systems

CAD systems are computer based tools that help medical personnel in analysis of imaging studies and help in making better diagnostic decisions. CAD does not intend to replace doctors and instead, acts as a second opinion by identifying suspicious areas in medical images and providing useful information to support diagnosis. The final diagnosis is always made by the doctor[9].

2.2.1 Evolution of CAD Systems

The modern CAD framework was defined in the 1980s when the University of Chicago shifted the objective from automated diganosis to acting as a second reader [9]. Early iterations relied on manual image processing using edge detection, filtering, and feature extraction to locate nodules in chest X-rays or microcalcifications in mammograms. Chan et al. provided a pivotal study showing that radiologists using CAD identified more clustered microcalcifications than those working alone, despite a high rate of false positives [10].

Over the 1990s and 2000s, CAD spread to areas such as lung cancer detection on chest X-rays and CT scans, detection of vertebral fractures linked to osteoporosis, brain aneurysm detection using MR angiography, and tracking interval changes in bone scans [9]. A prospective study on mammography, covering tens of thousands of patients, showed a steady rise in cancer detection rates using CAD [11]. This included finding small invasive tumors earlier. Sensitivity for microcalcification clusters rose from roughly 87% in the early 1990s to about 98% by the mid-2000s, while false positives dropped [12]. This "classical CAD" era relied on statistical pattern recognition and manually engineered features. Deep learning has since become the dominant approach in CAD.

The purpose of CAD has also shifted from early tools focused on detection by flagging suspicious zones, to later versions, which added differential diagnosis to estimate if a nodule was malignant or benign. Some systems even retrieve similar past cases from hospital Picture Archiving and Communication Systems (PACS) for comparison. Observer studies indicate that likelihood-of-malignancy scores improve radiologist decisions in subtle cases. Doctors still rely on their own judgment for obvious cases while CAD provides the most utility in borderline situations [13].

Future CAD development will likely move away from stand-alone tools. Instead, these will be integrated packages within PACS, allowing a single scan to be screened for multiple conditions simultaneously [14]. Deep learning has sped up this process.

A single trained model can often be adapted for various abnormalities with far less manual effort than previous systems required.

2.3 Deep Learning Approaches

The shift from classical CAD methods to deep learning occurred for a few reasons. Firstly, deep learning models such as CNNs and ViTs have the ability to learn specific image features from pixel data directly rather than having to manually design features and rules for all types of lesions. This makes deep learning models more flexible and often more accurate across different diseases and imaging types. Secondly, the emerging of large labeled medical imaging datasets and the availability of powerful GPU computing have made it possible to train these deep networks effectively. Thirdly, deep learning networks are more robust to image quality, patient anatomy and scanner types compared to rule based systems. Finally, they continuously outperform classical CAD methods in pivotal studies. The subsequent sections review some of the popular deep learning models used in medical imaging research, and also used in this study.

2.3.1 nnU-Net

The nnU-Net addresses a persistent problem in biomedical image segmentation. Even well-established architectures like the U-Net require extensive, dataset-specific manual tuning of preprocessing, network topology, and training parameters. Small misconfigurations can cause large performance drops. Rather than proposing a new architecture, nnU-Net systematizes this configuration process itself. It decomposes design decisions into three categories:

1. Fixed parameters that generalize across datasets (e.g., architecture template, loss function, optimizer).

2. Rule-based parameters derived automatically from a "dataset fingerprint" (image size, spacing, intensity distribution, modality) via heuristics (e.g., target spacing, patch size, batch size, network depth).

3. A small set of empirical parameters selected via cross-validation (choice among 2D, 3D, and 3D cascade configurations, and whether to apply post-processing such as largest-component suppression).

Figure 2.2 shows the nnU-Net configuration method:

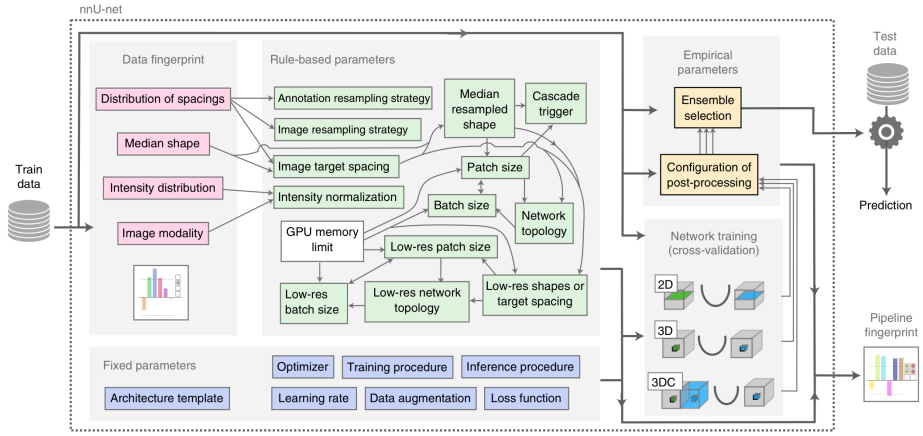

Figure 2.2: nnU-Net configuration method

Evaluated across 23 public datasets and 53 segmentation tasks spanning MRI, CT, electron microscopy, and fluorescence microscopy, nnU-Net outperformed most specialized, task-tuned pipelines without any manual intervention, setting a new state of the art on 33 of 53 target structures. Notably, the authors' analysis of the KiTS 2019 leaderboard showed that configuration choices such as patch size, spacing, normalization affected performance more than architectural modifications. Figure 2.3 shows example segmentations produced by nnU-Net on different datasets:

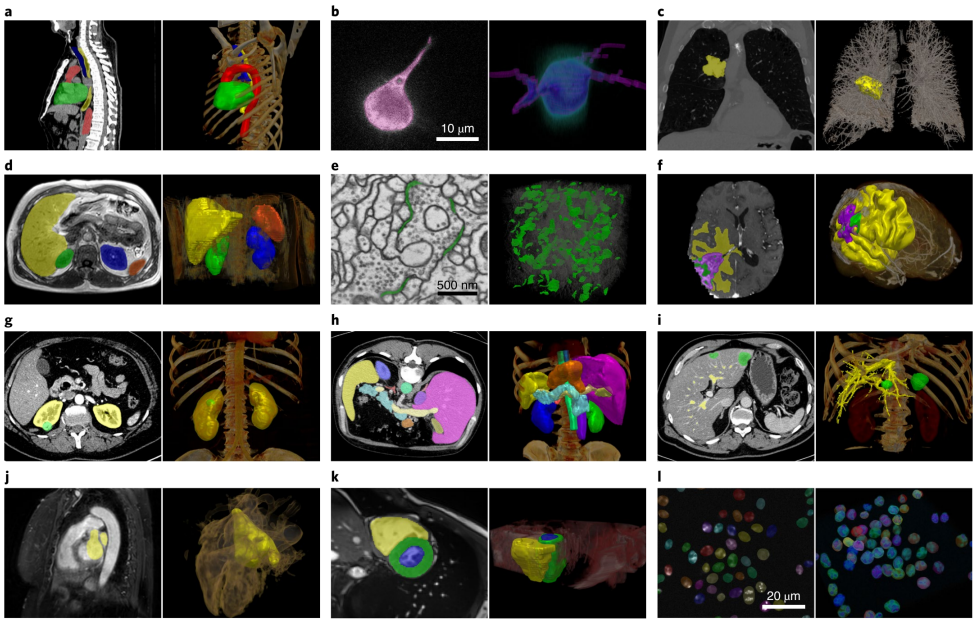

Figure 2.3: Example segmentations by nnU-Net

This suggests that careful pipeline configuration is often the primary bottleneck in medical image segmentation. This positions nnU-Net less as a competing architecture and more as a strong, reproducible baseline and out-of-the-box framework, and it is frequently cited in the literature as the standard benchmark against which new segmentation methods are compared.

2.3.2 BioMedParse

BiomedParse is a "Foundation Model for Joint Segmentation, Detection, and Recognition of Biomedical Objects Across Nine Modalities" [15]. Rather than relying on a fixed label set during inference, the model accepts a natural language prompt describing the target anatomy or pathology and produces a pixel level mask for the queried concept. This makes BiomedParse especially useful in settings where the structures of interest are heterogeneous, difficult to enumerate in advance, or described differently across institutions and reports.

The model combines image encoding with language understanding and uses a transformer based decoder to align the prompt with the visual features extracted from the scan [15]. Figure 2.4 shows the flowchart of the architecture:

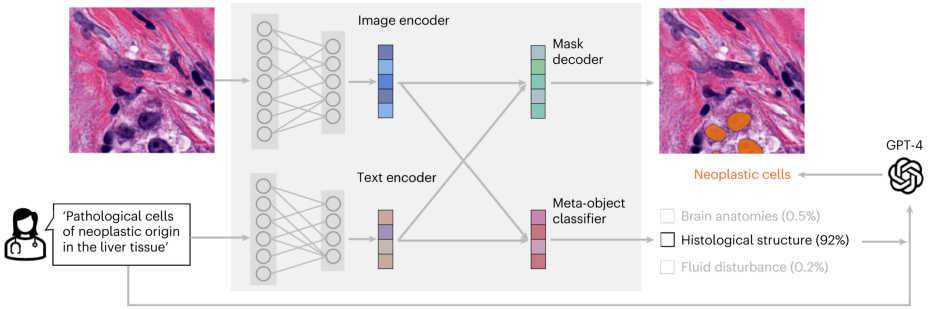

Figure 2.4: BiomedParse architecture

The output is a segmentation mask, as shown in Figure 2.5.

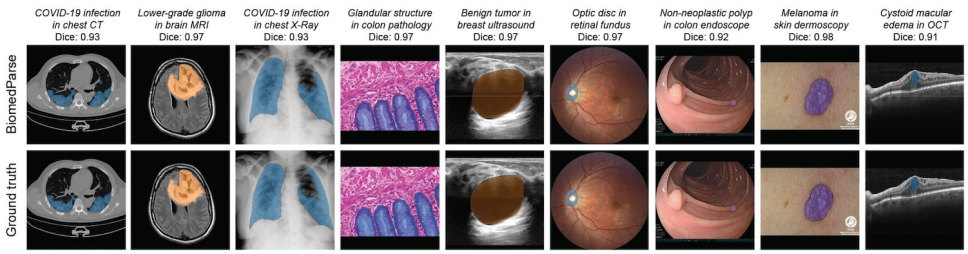

Figure 2.5: Example outputs of BiomedParse

BiomedParse serves as a SotA reference model for discussing localization quality, and interpretability in our study.

2.3.3 TotalSegmentator

Many deep learning methods were developed to segment only one or a few organs. Wasserthal et al. introduced TotalSegmentator, a deep learning framework that can automatically segment 104 anatomical structures from a single CT scan [16]. Figure 2.6 shows the main classes of TotalSegmentator, which include 27 organs, 59 bones, 10 muscles, 8 vessels.

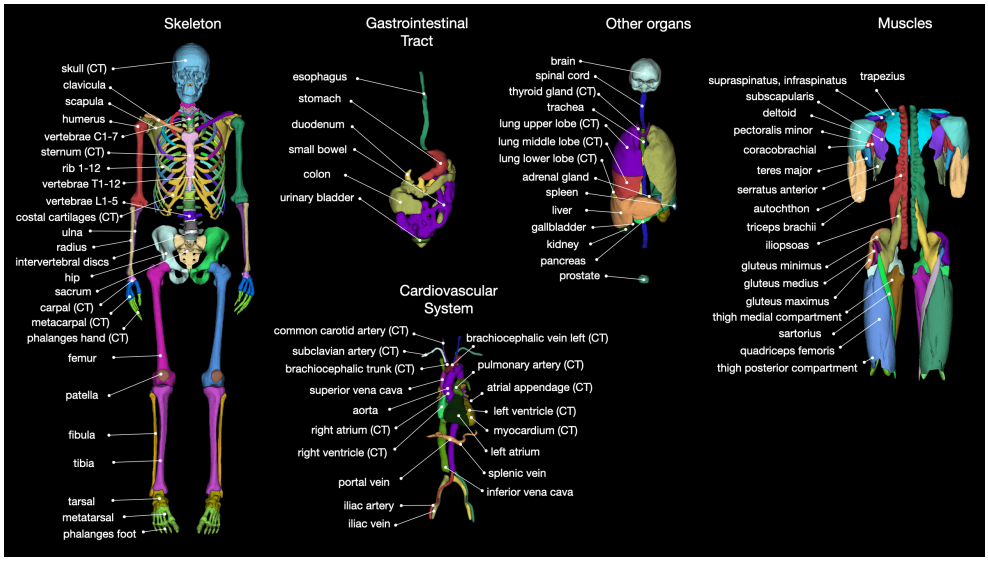

Figure 2.6: Main classes of TotalSegmentator

The framework is built on the nnU-Net architecture and was trained using more than 1,200 routine clinical CT scans collected from different hospitals and patient groups [16]. It achieved a mean Dice Similarity Coefficient (DSC) of about 0.94. Because of its high segmentation accuracy, TotalSegmentator is commonly used as a preprocessing tool in medical image research [17].

2.3.4 Merlin

Merlin is an AI model that can help interpret CT scans automatically. 3D vision-language model (VLM) built to understand abdominal CT scans by learning jointly from the images, radiology reports, and electronic health record (EHR) data [18].

Most earlier medical vision-language models were built for 2D images, such as chest X-rays, paired with short captions. CT scans are different, they are 3D volumes made of hundreds of slices, and simply applying a 2D model slice-by-slice throws away important 3D spatial information, like how a lesion connects across neighboring slices. Merlin was built to fix this by processing the full 3D CT volume directly, instead of treating it as a stack of unrelated 2D images [18].

Merlin is trained using over 6 million CT images from more than 15,000 scans, paired with over 1.8 million EHR diagnosis codes and more than 6 million tokens of radiology report text [18]. Instead of relying only on manually labeled data, which is slow and expensive to collect, Merlin learns from data hospitals already generate: doctors' reports and diagnosis codes. Merlin follows the concept introduced by CLIP, a foundational vision-language model designed originally for natural images. In CLIP style training, similar or matching image or text pairs closer to each other in a shared representation space while non-matching pairs are pushed apart making the model learn which images and text describe the same thing. Merlin adapts this idea to 3D CT data and adds EHR code supervision on top, which earlier medical VLMs generally did not use [18].

Merlin is part of a broader wave of work bringing vision-language modeling into 3D medical imaging, alongside similar efforts like CT-CLIP, which uses the same contrastive approach but focuses on chest CT scans[18]. Overall, Merlin shows that

learning directly from full 3D volumes and everyday hospital data, rather than small 2D datasets with manual labels, can produce useful for reducing radiologist workload.

2.3.5 MedSAM

MedSAM adapts the Segment Anything Model (SAM), originally developed by Meta AI for natural images, into a general-purpose segmentation model for medical scans [19]. Rather than training a separate network for each organ or modality, a single fine-tuned model is reused across many segmentation tasks. A key limitation is that MedSAM operates on 2D slices. Applying it to a 3D chest CT volume therefore requires a separate bounding box on every slice, and the resulting masks do not capture how a nodule connects across neighbouring slices.

MedSAM2 addresses this limitation by building on the SAM2 architecture and its memory-attention mechanism, allowing a single prompt to be propagated across an entire volume instead of being redrawn slice by slice, which substantially reduces the annotation effort required [20]. Specifically, that prompt is purely spatial, a box(two corner points), a point(roughly inside the target), or a mask(rough/full segmentation on that one slice) on one key slice, with no text or class label involved; from it, MedSAM2 segments the object on that slice and then propagates the mask forward and backward through the remaining slices via memory attention, without requiring a new prompt on each one.

Despite this improvement, pulmonary findings remain difficult for these models to segment accurately. A benchmarking study comparing MedSAM and MedSAM2 against standard segmentation networks found that both achieved strong performance on whole-lung segmentation but performed noticeably worse on individual tumours and small nodules, with results sensitive to the quality and consistency of the box prompts [21]. This reflects a broader limitation of prompt-driven segmentation models.

This sensitivity to prompt quality points to a more basic problem: neither MedSAM nor MedSAM2 can be told *what* to segment in words, simply because the SAM/SAM2 backbone they build on has no language pathway, only a geometric one. Meta's SAM 3 is aimed exactly at this gap through what it calls *Promptable Concept Segmentation*. Specifically, that prompt is purely spatial, a box(two corner points), a point(roughly inside the target), or a mask(rough/full segmentation on that one slice)t [22]. MedSAM-3 takes this a step further and fine-tunes the architecture on medical images paired with concept phrases, things like "lung infection" for chest X-ray or "pulmonary artery" for 3D lung CT [23]. Even so, the authors' own ablations make clear that text alone does not remove the need for a geometric anchor. On the 3D Parse2022 lung CT benchmark, raw SAM 3 given nothing but the phrase "pulmonary artery" managed a Dice of only 0.53, well short of nn-U-Net's 0.83; on 2D benchmarks such as BUSI, text-only prompting scored 0 for SAM 3 and just 0.27 after MedSAM-3 fine-tuning, jumping to 0.78 only once a bounding box was added alongside the text [23]. It is also worth noting that MedSAM-3's fine-tuning was limited to four 2D datasets, and its 3D lung CT evaluation went no further than the untuned SAM 3 baseline on pulmonary artery delineation; neither this paper nor the wider MedSAM family reports results on lung nodule or ground-glass-opacity segmentation in chest CT.

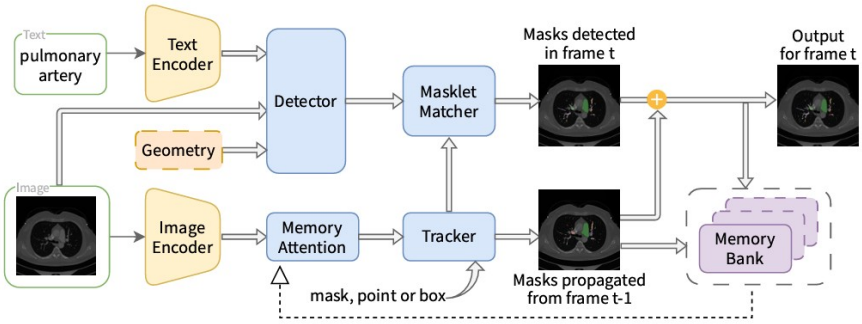

Figure 2.7: Architecture of MedSAM-3

2.3.6 CT-CLIP

CT images contain valuable information for diagnosing pulmonary diseases and are commonly used in many Deep Learning approaches as a single source of truth. However, radiology reports also provide detailed clinical descriptions of the findings in these CT images. Hamamci et al. proposed Computed Tomography Contrastive Language-Image Pre-training (CT-CLIP) [24], a vision-language foundation model for chest CT images, which utilizes both modalities. Unlike traditional supervised models that require large amounts of labeled data, CT-CLIP learns the relationship between CT scans and their corresponding radiology reports using contrastive learning. This allows the model to learn meaningful image and text representations without task-specific annotations.

CT-CLIP consists of a 3D image encoder and a text encoder. The image encoder extracts features from CT volumes, while the text encoder processes the corresponding radiology reports. Figure 2.8 shows the architecture of CT-CLIP.

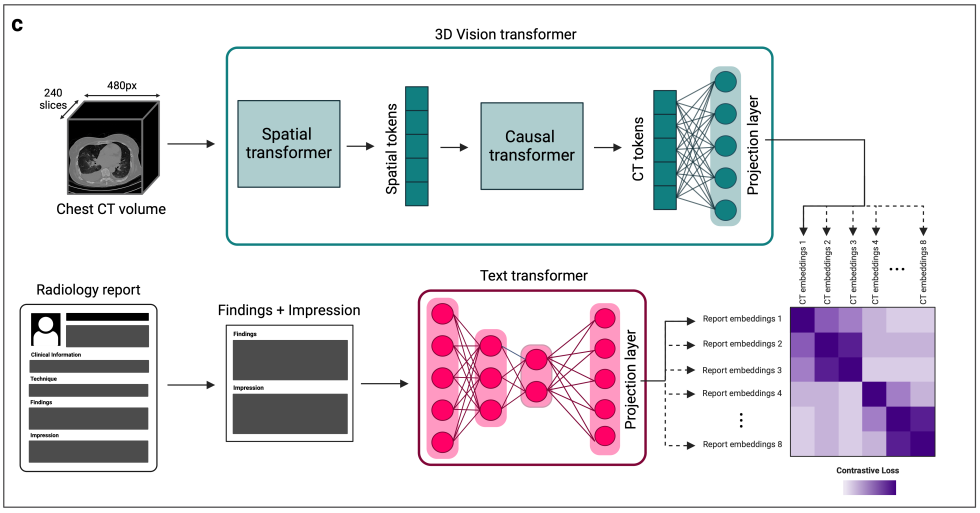

Figure 2.8: CT-CLIP architecture

During training, the model learns to match each CT scan with its correct report in a shared feature space. The model was trained using the CT-RATE dataset [25], which pairs over 50,000 reconstructed 3D chest CT volumes from around 25,700 scans of over 21,000 patients with their corresponding radiology reports. It has shown

strong performance in pulmonary disease classification, image retrieval, and zero-shot learning tasks, outperforming fully supervised models across key metrics [24].

2.4 Explainable AI (XAI) in Medical Imaging

Many deep learning models achieve high accuracy, but their decision-making process is often difficult to interpret and are referred to as "black-box" models [26]. In healthcare, this lack of transparency can reduce clinicians' confidence in AI-assisted diagnosis, as medical decisions directly affect patient outcomes. XAI methods aim to solve this issue.

2.4.1 LIME

Local Interpretable Model-agnostic Explanations (LIME), proposed by Ribeiro et al. [27], is a model-agnostic technique for explaining the predictions of any classifier or regressor by locally approximating its behavior with an interpretable surrogate model. The core idea is to treat the original model f as a black box and, for a given prediction on instance x, learn a simple explanation model g (typically a sparse linear model) that is faithful to f in the vicinity of x, even if g cannot approximate f well globally. Formally, LIME finds an explanation by minimizing an objective:

$$\xi(x) = \arg\min_{g} L(f, g, \pi_x) + \omega(g)$$

where L measures how unfaithful g is to f within a local neighborhood weighted by a proximity function π_x, and $\Omega(g)$ penalizes the complexity of the explanation to keep it human-interpretable. To ensure explanations are understandable regardless of the model's actual features, LIME operates over an interpretable representation such as a set of super-pixels for images—rather than the model's raw internal features (pixel tensors).

Through simulated experiments and studies, the authors demonstrate that LIME explanations are faithful to the underlying model, recovering ¿90% of known important features [27]. It helps users select the better-generalizing classifier between two competing models even when held-out accuracy is misleading, enables non-experts to perform effective feature engineering to improve an untrustworthy classifier, and allow users to detect models that rely on spurious correlations.

2.4.2 SHAP

SHapley Additive exPlanations (SHAP) is a unified framework for interpreting the predictions of complex machine learning models, proposed by Lundberg and Lee [28]. It builds on the observation that many existing interpretability methods—including Local Interpretable Model-agnostic Explanations (LIME), Deep Learning Important FeaTures (DeepLIFT), layer-wise relevance propagation, and classical Shapley value approaches—can all be framed as instances of a single class called additive feature attribution methods, in which an explanation model g approximates the original model f as a linear function of binary "presence/absence" indicator variables:

$$g(z') = \phi_0 + \sum \phi_i z'_i$$

Within this class, the authors show that only one explanation satisfies three desirable properties simultaneously:

1. Local Accuracy - The explanation must sum to match the model's actual output.
2. Missingness - Features absent from the input receive zero attribution.
3. Consistency - A feature's attributed importance cannot decrease if the model comes to rely on it more, holding other inputs fixed.

This unique solution fairly distribute a prediction's output among input features by averaging each feature's marginal contribution across all possible orderings/subsets of the other features—giving rise to the name SHAP values.

Because exact Shapley value computation is combinatorially expensive, the paper introduces several practical estimation methods. Kernel SHAP is a model-agnostic approach that computes SHAP values via specially weighted linear regression, connecting the LIME framework to Shapley values by deriving the specific loss function, weighting kernel, and regularization needed to make LIME's regression-based estimate satisfy the above three properties [28].

For deep neural networks specifically, the authors introduce Deep SHAP, which combines the compositional, layer-by-laye propagation style of DeepLIFT with Shapley value theory to more efficiently approximate feature attributions, and Max SHAP, which computes Shapley values for max-pooling operations in polynomial rather than exponential time. Empirically, the authors demonstrate that SHAP values align more closely with human-generated explanations than LIME or DeepLIFT, and that Kernel SHAP achieves comparable accuracy to sampling-based Shapley estimation with substantially fewer model evaluations, establishing SHAP as a theoretically grounded and practically efficient tool for model interpretability [28].

2.4.3 Grad-CAM

Gradient Class Activation Mapping (Grad-CAM), proposed by Selvaraju et al. [29], is a technique for producing visual explanations from CNNs by using gradient information to identify which regions of an input image are important for a given prediction. Unlike its predecessor, Class Activation Mapping (CAM), Grad-CAM works with any CNN-based architecture without changes or retraining. The method computes the gradient of a target class score with respect to the feature maps of the last convolutional layer, then global-average-pools these gradients across the spatial dimensions to obtain a set of neuron importance weights α_k^c. A weighted combination of the forward activation maps using these weights are passed through a ReLU to retain only features with a positive influence on the target class. This produces a coarse heatmap highlighting class-discriminative regions of the image. The authors formally prove that Grad-CAM is a strict generalization of CAM, reducing to the same weights when applied to CAM-compatible architectures.

Through extensive evaluation, the authors demonstrate that Grad-CAM achieves superior weakly-supervised localization performance compared to prior methods without sacrificing classification accuracy [29]. They further showed that Grad-CAM's localizations correlate more strongly with occlusion-based faithfulness measures than competing visualization techniques, indicating it is more faithful to the underlying model. Beyond diagnostic use for uncovering CNN failure modes and robustness to

adversarial perturbations, the authors demonstrate Grad-CAM's utility for identifying and mitigating dataset bias, famously showing that a biased "doctor vs. nurse" classifier had learned to rely on gender-correlated cues such as hairstyle, rather than clinically relevant features.

2.4.4 Other CAM Methods

Grad-CAM++

Chattopadhyay et al. propose Grad-CAM++ [30] as a generalization of Grad-CAM that addresses two of its key limitations:

1. Degraded localization when multiple instances of the same class appear in a single image

2. Incomplete coverage of an object's extent in single-object images

While gradient-based methods like Grad-CAM provide fine-grained explanations, their performance drops when localizing multiple occurrences of the same class, and Grad-CAM heatmaps often fail to capture the entire object [30]. Grad-CAM++ uses a weighted combination of the positive partial derivatives of the last convolutional layer's feature maps with respect to a specific class score as pixel-wise weights, producing a visual explanation for the class of interest. By deriving closed-form higher-order gradient weighting coefficients rather than Grad-CAM's simple global-average-pooled gradients, Grad-CAM++ generates explanation maps that better localize multiple object instances and cover objects more completely.

Score-CAM

Score-CAM proposed by Wang et al. [31] removes the dependence on gradients entirely. Unlike previous CAM approaches, Score-CAM obtains the weight of each activation map through its forward-passing score on the target class rather than through gradients, with the final saliency map obtained as a linear combination of these weights and the activation maps. Concretely, each channel of the final convolutional layer's activation map is upsampled to input resolution and used as a mask over the original image; the resulting masked images are forward-passed through the network, and the change in the target class's confidence score serves as that channel's importance weight. Because it dispenses with backpropagated gradients—which can be noisy or saturate—Score-CAM was shown to produce better visual quality and fairness in explanation than gradient-based CAM variants, at the cost of requiring multiple forward passes.

LayerCAM

Jiang et al. introduced LayerCAM [32] to address the coarse spatial resolution of standard CAM/Grad-CAM maps, which are typically generated only from the final convolutional layer. Because such maps have low spatial resolution, they often only locate coarse regions of target objects, limiting performance on tasks needing pixel-accurate localization. By rethinking the relationship between feature maps and their corresponding gradients, LayerCAM produces reliable class activation maps at different layers of the CNN, using element-wise gradient weighting, rather than globally

pooled gradients, so that shallower layers—which retain finer spatial detail—can also contribute meaningful, class-discriminative localization. These layer-wise maps can then be fused into a single higher-resolution activation map, better capturing fine object boundaries than Grad-CAM's coarse, final-layer-only heatmap.

2.4.5 Limitations of XAI Methods

Despite substantial progress, XAI continues to face a fundamental accuracy-interpretability tradeoff: highly interpretable models such as decision trees, linear regression, and rule-based learners are intrinsically limited in their ability to model non-linear, high-dimensional relationships. Deep neural networks that achieve state-of-the-art performance remain the least interpretable [33]. Model extraction and distillation techniques, which attempt to approximate black-box behavior with a simpler surrogate model, partially address this but reduces fidelity to the original model's decision boundaries. Post-hoc explanation methods such as LIME offer only local fidelity and provide an unsatisfactory global approximation, and saliency maps such as Grad-CAM can produce noisy uninterpretable outputs. Furthermore, saliency maps assume that all model-relevant features are inherently human-interpretable, which does not always hold [34]. Explanation techniques are also vulnerable to manipulation and adversarial perturbation; LIME has been shown to develop self-learned patterns that produce misleading explanations under such conditions, raising concerns about the robustness and trustworthiness of the interpretations themselves [34].

Beyond technical constraints, XAI suffers from a lack of standardized, universally accepted evaluation frameworks, since interpretability and explainability are frequently conflated and no consensus exists on how to rigorously quantify explanation quality [35, 36]. Existing evaluation paradigms: application-grounded, human-grounded, and functionally grounded, each carry tradeoffs between cost, human involvement, and validity. The subjective, user-dependent nature of what constitutes a "good" explanation makes benchmarking especially difficult [33]. Additional challenges include the computational expense of methods like Model Class Reliance, the difficulty such methods have scaling to complex, high-dimensional, or strongly feature-correlated models, and the broader absence of a unified theoretical framework connecting the disparate strands of explanation research [37].

2.5 Sparse Coding

Why is sparsity useful?

How is sparse representation learned?

Sparse coding is a signal representation framework in which a signal is expressed as a linear combination of only a small number of elements ("atoms") drawn from a dictionary, rather than as a dense combination over an orthogonal basis. Formally, given a signal x and a dictionary D whose columns are atoms, sparse coding seeks a coefficient vector α such that:

$$x \approx D\alpha,$$

with α containing mostly zero entries. A key aspect of this framework is the use of overcomplete representations, where the dictionary D has more atoms than the

dimensionality of the signal itself (i.e., more columns than rows). This redundancy is deliberate: it gives the representation greater flexibility to match the underlying structure of the signal, allowing different signals to be well-approximated by different small subsets of atoms, which improves representational power, robustness to noise, and the ability to capture diverse structural patterns compared to a complete (non-redundant) basis. The trade-off is that recovering α becomes an underdetermined and generally NP-hard combinatorial problem, since many possible coefficient vectors could reconstruct x, and one must impose sparsity as a constraint or penalty to select a meaningful solution.

Because exact sparsest-solution search is computationally intractable, two main families of practical algorithms are used:

1. Greedy Algorithms (Matching Pursuit, OMP)
2. Convex relaxation Algorithms (LARS, FISTA)

Greedy methods iteratively select the atom most correlated with the current residual, add it to the active support set, and update the coefficients, repeating until a stopping criterion (sparsity level or residual error) is met; these methods are fast and simple but only approximate the optimal solution.

Convex relaxation methods instead replace the intractable $\ell 0$-norm (which counts nonzero entries) with the $\ell 1$-norm, yielding a convex optimization problem—as in Basis Pursuit or the Lasso—that can be solved reliably and, under certain conditions on the dictionary (e.g., restricted isometry or incoherence properties), provably recovers the same sparse solution as the original $\ell 0$ problem. In practice, greedy methods are often preferred for speed and low-sparsity regimes, while convex relaxation approaches tend to offer stronger theoretical guarantees and more stable solutions in noisier or higher-sparsity settings.

2.5.1 Matching Pursuit (MP)

Matching Pursuit is a greedy algorithm for approximating a signal x as a sparse combination of atoms from a dictionary D. It proceeds iteratively: at each step, it computes the inner product between the current residual (initially the signal itself) and every atom in the dictionary, selects the atom with the highest absolute correlation, and updates the coefficient for that atom by projecting the residual onto it. The residual is then updated by subtracting this contribution, and the process repeats until a stopping criterion is reached (target sparsity or residual energy threshold). MP is simple and computationally cheap, but because it only updates one coefficient per iteration without adjusting previously selected coefficients, it can select the same atom multiple times and converges slowly, and its solutions are generally suboptimal compared to more refined methods.

2.5.2 Orthogonal Matching Pursuit (OMP)

OMP improves on MP by re-optimizing all currently selected coefficients at every iteration rather than only the most recently added one. At each step, it selects the atom most correlated with the residual (as in MP), adds it to the active support set, and then solves a least-squares problem over all atoms currently in the support to

obtain new coefficients that minimize the residual jointly. Because the residual is then orthogonal to all previously selected atoms, OMP never reselects the same atom twice, and it typically converges in fewer iterations and yields more accurate reconstructions than plain MP. The added cost is the least-squares solve at each iteration, making OMP more computationally expensive per step, though still tractable for moderate sparsity levels.

2.5.3 Least Angle Regression (LARS)

LARS is an efficient algorithm for computing the entire regularization path of the Lasso ($\ell 1$-penalized least squares) problem. Rather than fixing a penalty parameter λ and solving a single optimization, LARS starts with all coefficients at zero and iteratively identifies the predictor (atom) most correlated with the current residual, moving the coefficient in that predictor's direction. Instead of moving all the way to the least-squares solution as in stepwise regression, LARS moves only until another predictor becomes equally correlated with the residual, at which point that predictor is added to the active set and the direction is updated to remain equiangular between all active predictors. With a simple modification (allowing coefficients to be dropped from the active set when they cross zero), LARS exactly reproduces the full Lasso solution path with a computational cost comparable to ordinary least squares, making it a popular and efficient tool for convex-relaxation-based sparse coding, particularly when the full path of solutions across sparsity levels is of interest.

2.5.4 Fast Iterative Shrinkage-Thresholding Algorithm (FISTA)

FISTA solves $\ell 1$-regularized sparse coding problems, where the goal is to minimize:

$$\sqrt{\|x - D\alpha\|}^2 + \lambda\|\alpha\|_1 \tag{2.1}$$

It builds on the Iterative Shrinkage-Thresholding Algorithm (ISTA), which alternates between a gradient descent step on the data-fidelity term and a soft-thresholding (shrinkage) step that enforces sparsity by shrinking small coefficients toward zero. While ISTA converges at a sublinear rate of $O(1/k)$, FISTA accelerates this by incorporating a momentum term—extrapolating using a weighted combination of the current and previous iterates—which improves the convergence rate to $O(1/k^2)$ without increasing per-iteration cost. This makes FISTA well-suited to large-scale sparse coding and compressed sensing problems where fast, reliable convergence to the $\ell 1$-regularized solution is needed.

2.6 Dictionary Learning

Dictionary learning is a technique in which an image patch or a region of a medical scan is represented as a combination of a small number of basic building blocks, rather than by describing every pixel value directly. This idea comes from sparse coding theory and is widely used in image denoising, compression, and interpretable machine learning, including medical image analysis [38] [39].

A dictionary is a matrix, written as $D \in \mathbb{R}^{n\times k}$, where each column d_i is called an atom. Each atom is a small, fixed pattern, for example a short edge, a texture, or a simple shape, that acts like a basic building block for describing images [38].

$$D = [\, d_1, d_2, \ldots, d_k \,], \qquad \|d_i\|_2 = 1 \ \forall i \tag{2.2}$$

Each atom is normalized to unit length so that none of them dominate because of scale. A good dictionary has atoms that are simple and reusable.

Once a dictionary is available, any image $x \in \mathbb{R}^n$ is approximated as a weighted sum of a small number of atoms:

$$x \approx D\alpha = \sum_{i=1}^{k} \alpha_i d_i \tag{2.3}$$

Here, $\alpha \in \mathbb{R}^k$ is the sparse code; most of its values are zero, and only a few atoms are actually used to rebuild the image. Finding this code is called sparse coding, and it can be solved by minimizing the reconstruction error together with a sparsity penalty [39] [40]:

$$\min_{\alpha} \ \frac{1}{2}\|x - D\alpha\|_2^2 + \lambda\|\alpha\|_1 \tag{2.4}$$

This is the Lasso formulation, where λ controls how sparse the solution is [41]. When the dictionary itself is learned from training data $X = [x_1, \ldots, x_m]$, both D and the codes $A = [\alpha_1, \ldots, \alpha_m]$ are optimized together

$$\min_{D,A} \ \|X - DA\|_F^2 + \lambda\|A\|_1 \quad \text{subject to} \quad \|d_i\|_2 = 1 \tag{2.5}$$

Learned atoms are not random they turn out to look like meaningful patterns. Scince each image patch is reconstructed using only a small number of atoms. Each atoms tends to specialize in representing a simple pattern rather than blending many patterns together. This lets a person actually look at a lerned dictionary and understand what each atom represents, wich is must harder with internal weights of a deep neural network. In medical imaging, atoms can end up representing specific tissue textures or lesion patterns that a radiologist can vissually recognize[42].

Learning a dictionary is usually done by fixing the dictionary and solving for sparse codes, then fixing the codes and updating the dictionary. The sparse coding step is commonly solved with Orthogonal Matching Pursuit (OMP), a greedy method that adds one atom at a time [43]

$$i^* = \arg\max_i \ |d_i^T r|, \qquad r = x - D\alpha \tag{2.6}$$

or with Lasso/LARS, using the ℓ_1-penalized objective shown above [41]. The dictionary update step is where the main algorithms differ, covered below.

2.6.1 K-Means Singular Value Decomposition (K-SVD)

K-SVD is one of the most widely used algorithms for learning a dictionary, and it solves the following optimization problem[41]

$$\min_{D,X} \left\{ \|Y - DX\|_F^2 \right\} \quad \text{subject to} \quad \forall i, \ \|x_i\|_0 \le T_0 \tag{2.7}$$

Here, Y is the matrix of training data, for example CT image patches, D is the dictionary being learned, and X is the matrix of sparse codes, where each column x_i is the code for one training sample. The constraint $\|x_i\|_0 \le T_0$ means that each

code is allowed to use at most T_0 atoms, using the ℓ_0 norm to directly count non-zero entries rather than approximating sparsity with an ℓ_1 penalty like Lasso does. This gives K-SVD tighter control over exactly how sparse each representation is.

K-SVD solves this problem by alternating between two steps. First, with the dictionary D fixed, it finds the sparse codes X using a pursuit algorithm such as Orthogonal Matching Pursuit. Second, with the codes fixed, it updates the dictionary one atom at a time instead of all at once, which makes it converge faster and produce sharper, more specific atoms [41].

2.6.2 Method of Optimized Directions (MOD)

Method of Optimized Directions is an earlier and simpler approach than K-SVD. Instead of updating atoms one at a time, MOD updates the entire dictionary at once, using a direct least-squares solution [44]. Given the training data X and current sparse codes A, the optimal dictionary is:

$$D = XA^T(AA^T)^{-1} \tag{2.8}$$

This comes directly from setting the derivative of the reconstruction error $\|X - DA\|_F^2$ to zero and solving for D. MOD is fast and easy to implement, but because it updates all atoms together, it tends to need more iterations to converge compared to K-SVD, and can get stuck producing less sharp atoms [44].

2.6.3 Online Dictionary Learning (ODL)

Both K-SVD and MOD assume the entire training dataset fits in memory and is processed all at once, which becomes impractical for very large image datasets. Online Dictionary Learning solves this by processing the data in small batches, updating the dictionary incrementally as new samples arrive, instead of waiting to see the whole dataset [45]. At each step t, a new sample x_t is drawn, its sparse code α_t is computed using the current dictionary, and then the dictionary is updated using accumulated statistics from all samples seen so far:

$$A_t = A_{t-1} + \alpha_t \alpha_t^T, \qquad B_t = B_{t-1} + x_t \alpha_t^T \tag{2.9}$$

$$D_t = \arg\min_D \frac{1}{t} \sum_{i=1}^{t} \left(\frac{1}{2} \|x_i - D\alpha_i\|_2^2 \right) \tag{2.10}$$

which can be solved efficiently using A_t and B_t instead of storing all past samples. This makes ODL well suited for large-scale problems, including learning dictionaries from thousands of medical image patches, since memory use stays constant regardless of dataset size [45].

2.7 Discriminative Dictionary Learning

Traditional dictionary learning methods mainly focus on minimizing the reconstruction error between the input data and its sparse representation [46]. However, good reconstruction does not necessarily produce features that are useful for classification.

Samples from different classes may activate similar atoms if the learned dictionary only captures common structures in the data.

Discriminative dictionary learning addresses this limitation by incorporating class information during dictionary optimization. The objective is not only to represent the input samples accurately but also to learn sparse codes that improve the separation between different classes. By learning class aware atoms, discriminative dictionary learning can provide both classification capability and a more interpretable representation. Several approaches have been proposed, including Label Consistent K-SVD (LC-KSVD) and Fisher Discriminative Dictionary Learning (FDDL), which extend the K-SVD framework by introducing supervised learning constraints.

2.7.1 Label Consistent K-SVD (LC-KSVD)

Label Consistent K-SVD (LC-KSVD), introduced by Jiang et al. [47], extends the classic K-SVD framework by embedding label consistency into the dictionary update process. The core idea is to encourage sparse codes from samples of the same class to exhibit similar activation patterns, while those from different classes diverge. This is achieved without sacrificing the algorithm's efficient iterative structure involving sparse coding and dictionary refinement.

The method comes in two main variants. LC-KSVD1 incorporates a label consistency term that aligns the sparse codes with an ideal discriminative structure through a linear transformation:

$$\min_{D,A,A_c} \|X - DA\|_F^2 + \alpha\|Q - A_cA\|_F^2 \quad \text{s.t.} \quad \|a_i\|_0 \leq T \ \forall i. \tag{2.11}$$

Here, Q is a target matrix encoding class specific indicator patterns, and A_c learns to map codes toward this structure. LC-KSVD2 extends LC-KSVD1 by adding a classifier learning term. The dictionary, sparse codes, label consistency transformation, and classifier are jointly optimized:

$$\min_{D,A,A_c,W} \|X - DA\|_F^2 + \alpha\|Q - A_cA\|_F^2 + \beta\|H - WA\|_F^2 \quad \text{s.t.} \quad \|a_i\|_0 \leq T, \tag{2.12}$$

where H contains the label information. The additional classification term allows the learned sparse representation to be directly optimized for prediction. Due to its ability to learn class specific atoms, LC-KSVD has been applied in image classification tasks where interpretability of learned features is important [47].

2.7.2 Fisher Discriminative Dictionary Learning (FDDL)

Fisher Discriminative Dictionary Learning (FDDL), proposed by Yang et al. [48], is another supervised dictionary learning approach that directly enforces Fisher like criteria on both the dictionary atoms and the sparse coefficients. It learns a structured dictionary $D = [D_1, D_2, \ldots, D_c]$ with class specific sub dictionaries D_i while promoting sparse codes A that exhibit low within class scatter and high between class scatter.

The core objective combines a discriminative fidelity term $r(\cdot)$ (ensuring good reconstruction within class and poor reconstruction across classes) with a Fisher discrimination penalty on coefficients:

$$\min_{D,X} \sum_i r(A_i, D, X_i) + \lambda_1 \|X\|_1 + \lambda_2 \Big(\operatorname{tr}(S_W(X)) - \operatorname{tr}(S_B(X)) + \eta \|X\|_F^2 \Big), \quad (2.13)$$

where $r(\cdot)$ encourages each class to be well represented by its own sub dictionary while poorly represented by others. S_W and S_B denote the within class and between class scatter matrices of the sparse codes. The elastic term controlled by η is added to guarantee that the overall Fisher discrimination function remains convex, which greatly improves optimization stability [48].

Compared with LC-KSVD, which introduces label consistency through a learned transformation and classifier, FDDL focuses on improving the statistical distribution of sparse codes. This makes it suitable for classification tasks where the separation between classes is important.

2.7.3 Frozen Dictionary Learning

While most discriminative methods update the entire dictionary during training, certain scenarios benefit from preserving previously acquired knowledge. Frozen dictionary learning, as introduced in the Outlier Learning via Augmented Frozen (OLAF) Dictionaries framework [49], addresses this by fixing a subset of atoms learned from baseline (normal) data while allowing new atoms to adapt to anomalous samples.

The composite dictionary takes the partitioned form $D = [D_f \mid D_u]$, where D_f remains frozen after initial training on normal data, and only the unfrozen portion D_u is updated on data containing deviations. Both K-SVD and Alternating Minimization algorithms can be modified straightforwardly to respect this constraint by skipping updates on frozen columns during the dictionary refinement stage [49].

This approach is particularly useful when preserving existing knowledge is important. In medical imaging, a dictionary learned from normal anatomy can be kept fixed, while additional atoms are learned from abnormal cases. This separation helps maintain normal structural information and improves the interpretability of disease related atoms. However, the effectiveness of this method depends on the quality of the initial frozen dictionary, since an incomplete initial representation may limit the ability of the model to adapt to new patterns.

2.8 Dictionary Learning in Medical Imaging

Dictionary learning has become a powerful tool across the medical imaging pipeline, offering data-adaptive alternatives to fixed transforms such as DCT or wavelets by learning sparse representations directly from image patches [50]. We have identified that DL-based methods are applied to medical imaging in 5 key aspects:

1. Signal Denoising
2. Image Reconstruction
3. Super-Resolution
4. Lesion Detection and Classification
5. Segmentation

2.8.1 Signal Denoising

In denoising, DL-based methods learn a dictionary of atoms from the noisy image itself, then reconstruct a clean image by enforcing that each local patch be sparsely represented by these learnt atoms. This approach has been applied to remove Gaussian noise from natural images [51]. In medical imaging, sparse denoising in diffusion MRI has shown to outperform standard DCT/FFT-based denoising by adapting to the specific structure of the data [52]. Shi et al. proposed an artifact suppressed dictionary learning algorithm (ASDL) [53], which exploited scale and orientation to build discriminative dictionaries for artifact suppression in LDCT.

2.8.2 Image Reconstruction and Super-Resolution

In reconstruction, DL has addressed the inverse problem of recovering images from undersampled k-space acquisitions in compressed sensing MRI [54]. Patch-based dictionaries provide a sparser and more flexible representation than global transforms like wavelets or temporal Fourier bases, enabling more accurate recovery from highly undersampled data.

Closely related to reconstruction is super-resolution, where coupled high-resolution and low-resolution dictionaries are co-trained so that corresponding patches share the same sparse code. This allows high-resolution image patches to be hallucinated from low-resolution inputs. This has been used to enhance cardiac MRI slabs and single-image brain MRI without requiring accurate subpixel alignment [55].

2.8.3 Detection, Classification and Segmentation

Since standard reconstructive dictionaries lack discriminative power, DDL techniques can be used for detection, classification and segmentation. DDL incorporates class labels during training, either by learning separate class-specific dictionaries and using representation residuals for classification, or by jointly learning a shared dictionary alongside a linear classifier over the sparse coding coefficients [50].

Class-specific dictionary learning has been directly applied to CAD for cancer detection. Liu et al. [56] proposed a sparse representation-based classification framework for colorectal polyp and lung nodule detection. The approach learned separate overcomplete dictionaries per class via K-SVD and classifying test samples either by the class whose dictionary yields the most non-zero coefficients when pooled into a global dictionary, or by the class-specific dictionary that achieves the sparsest reconstruction. This generative approach naturally extended to multi-class problems and, when fused with a relevance vector machine baseline through a gated decision tree, achieved consistent sensitivity gains at clinically acceptable false-positive rates on large multi-site colon and lung datasets.

Wu et al. [57] extended class-specific dictionary learning to pulmonary nodule detection in CT, addressing the practical problem that accurate nodule segmentation is often unreliable due to irregular shapes and low contrast. Rather than relying on precisely segmented ROIs, they learned two classification-oriented sub-dictionaries (nodule and non-nodule) together with a shared background dictionary, following the DL-COPAR formulation [58]. Sparse coefficients from the class-specific sub-dictionaries were pooled into feature vectors, refined using DX-score-based feature selection, and

passed to an SVM classifier, yielding good classification performance on the LIDC-IDRI dataset without requiring accurate nodule segmentation, and demonstrating that discriminative dictionary learning can substitute for precise localization as a prerequisite for feature extraction.

Ben-Cohen et al. [59] extended dictionary learning-based approaches from classification into a full lesion detection pipeline, combining a global fully convolutional network (FCN) that generates a lesion probability map across the CT volume with a local, discriminative dictionary-based module for false-positive reduction. Candidate regions from the FCN were partitioned into SLIC uperpixels, each represented by a 125-dimensional textural feature vector, and classified using LC-KSVD. A further innovation was online, per-case fine-tuning of the dictionary and linear classifier using high and low-confidence superpixels drawn from each test case's own FCN probability map, personalizing the discriminative dictionary without additional manual labels. On a clinical liver metastases dataset, this combined FCN-plus-sparsity framework achieved 94.6% sensitivity at 2.9 false positives per case, outperforming the FCN alone, U-Net, a superpixel-only sparse classifier, and a patch-based CNN, with particularly strong performance maintained on small lesions.

Al-Shaikhli et al. extended dictionary learning to a unified multi-class classification and multi-label segmentation framework for brain tumors. It is built around two distinct K-SVD dictionaries learned from a single Flair-MRI modality. A feature dictionary, learned from global topological features combined with gray-level co-occurrence texture features, is used for classification of the volume as normal or abnormal. If abnormal, it is further classified by finding which class-specific dictionary yields the sparsest representation of the test features. Separately, coupled data-label dictionaries are learned jointly from voxel-wise gray-scale patches and their corresponding ground-truth segmentation labels, so that each atom in the image dictionary has an associated atom in the label dictionary; at test time, sparse similarity between test patches and the data dictionary retrieves the matching label-dictionary atoms, which supply foreground/background costs for multi-label graph-cut segmentation of necrosis, edema, and enhancing/non-enhancing tumor. This decoupling of global topological/textural features for classification from local voxel-wise coupled dictionaries for segmentation, using only a single MRI channel, achieved 94.26% classification accuracy and segmentation performance competitive with or exceeding multi-channel BraTS-2013 methods requiring four MRI modalities.

Anitha's more recent hybrid framework similarly separates dictionary-based segmentation from a distinct classification stage, but replaces reconstruction-residual dictionary classification with a downstream regression-based classifier and shifts from pure MRI to fused 3D MRI-CT input: a Sparse Mahalanobis Dictionary learns class-discriminative atoms for tumor versus non-tumor tissue using a Mahalanobis-distance nonlinearity rather than plain Euclidean sparse coding. These dictionary-derived features feed a Multimodal Markov Random Forest that labels mutually exclusive and exhaustive tissue regions to avoid the tissue loss common in multi-decomposition fusion pipelines. The segmented tumor regions are then classified further by extracting nested 3x3 patches, computing Spearman-correlation-based co-occurrence features, and passing them to a Lagrangian neural network. On combined CT-hemorrhage and MRI tumor datasets this pipeline reported 99.7% accuracy, 98.9% sensitivity, and 98.7% specificity, outperforming CNN, CapsNet, ELM, and SVM-based baselines.

Wang et al. extended coupled dictionary learning to a multi-modality setting for nasopharyngeal carcinoma (NPC) segmentation, addressing the practical constraint that clinical datasets often mix multi-modality patients (with both CT and MRI) and single-modality patients. Building on local coordinate coding, their method jointly learns three coupled dictionaries (CT, MRI, and label) by sharing sparse coefficients across CT and MRI patches for multi-modality samples while separately fitting single-modality patches to their respective dictionary-label pairs, allowing all available data to contribute to a common sparse-coding space rather than discarding incomplete cases. At test time, a voxel is labeled by jointly encoding its CT and MRI patches against the learned CT/MRI dictionaries and projecting the resulting sparse coefficients onto the label dictionary. This joint approach outperformed single-modality dictionary learning (DSC of 76.87% vs. 65.36% for CT-only and 74.94% for MRI-only) and achieved segmentation accuracy comparable to prior specialized NPC methods.

Dong et al. proposed a simultaneous CT reconstruction and segmentation (SRS) framework that uses a discriminative dictionary pair, one for representing image intensity patches and one for representing the corresponding class-probability patches, learned jointly via an extension of K-SVD. The two dictionaries share the same sparse codes for each patch, so that during reconstruction the sparse coding step simultaneously constrains both the intensity reconstruction and the segmentation. The segmentation acts as a built-in regularizer for the otherwise ill-posed inverse problem. Their optimization alternates between sparse coding, an intensity reconstruction update solved via CGLS, and a class-probability update solved with a Frank-Wolfe algorithm, with total-variation regularization on the class probabilities to encourage piecewise-constant, smooth class interfaces. Tested on textured Brodatz-based phantoms and simulated fibrous material samples under varying numbers of projection angles, the method outperformed a non-dictionary SRS baseline and two weaker dictionary-based pipelines, with the gap widening as the number of projections decreased. The authors note the method's reliance on training data that adequately spans the class-interface geometries expected in the target images, since interfaces not represented in the dictionary tend to be oversmoothed or straightened in the output.

A related work applies DDL to the segmentation problem, once images are already reconstructed. Tong et al. extended their earlier hippocampus-labeling framework into a discriminative dictionary learning for segmentation (DDLS) method capable of handling multiple structures simultaneously. Their approach jointly learns a reconstructive dictionary and a linear classifier from patches extracted out of a pool of selected atlas images, so that the sparse code of a target patch can be mapped, via the learned classifier, directly to a probabilistic label vector. This produces a subject-specific probabilistic atlas that is then refined with a graph-cuts post-processing step. A central contribution is a voxel-wise local atlas selection strategy (L-DDLS), which selects different subsets of atlases at different spatial locations based on local similarity, rather than relying on one globally selected atlas set (G-DDLS) or a fixed offline-trained dictionary (F-DDLS). This addresses the high inter-subject anatomical variability of abdominal organs, particularly the pancreas, which is poorly served by whole-image similarity measures dominated by large organs like the liver. A multi-resolution Gaussian-pyramid scheme was also incorporated to keep the patch-based approach computationally tractable for large 3D organs. Evaluated on 150 abdominal CT scans, L-DDLS achieved Dice overlaps of 94.9%, 93.6%, 71.1%, and 92.5% for liver, kidneys, pancreas, and spleen respectively, consistently outperforming majority-

voting label fusion, non-local patch-based segmentation, and both G-DDLS and F-DDLS variants, with the largest relative gains for the anatomically variable pancreas. Notably, these results were obtained using only affine (not deformable) registration, underscoring how a learned discriminative dictionary can substitute for some of the burden typically placed on accurate non-rigid registration in multi-atlas segmentation pipelines.

Farhangi's doctoral thesis develops a shape-based segmentation framework for lung nodules in thoracic CT that relies on a dictionary of previously delineated nodule shapes rather than intensity patches. The method, called Sparse Combination of Training Shapes (SCoTS), embeds nodule boundaries as zero level sets of signed distance functions (SDFs) and builds a dictionary by vectorizing SDFs from a training set of expert-delineated nodules. The evolving segmentation surface is then approximated at each iteration as a sparse linear combination of these dictionary atoms, solved via an l1-penalized least-squares formulation using a coordinate-wise "shooting" algorithm. The key contribution is coupling this sparse shape prior with a Chan-Vese region-based active contour. The relative influence of the shape prior versus the low-level image-intensity energy is adjusted dynamically at each iteration based on the sparsity of the reconstruction coefficients, so that as the evolving surface approaches a plausible nodule shape and its representation becomes sparser, the shape prior is trusted more, preventing the contour from leaking into adjacent tissues of similar density. Validated on 542 3D nodules from the LIDC-IDRI database against four independent radiologists, SCoTS achieved a mean DSC of about 0.71-0.72 across radiologist delineations, with the primary failure mode occurring for nodules whose shapes were poorly represented in the training dictionary.

Discriminative dictionary learning has also been extended to multi-modal neuroimaging classification of Alzheimer's disease (AD) and mild cognitive impairment (MCI). Li et al. built on their previously introduced supervised within-class-similarity discriminative dictionary learning (SCDDL) algorithm by adding a within-class-similarity term to the standard discriminative dictionary objective, which penalizes deviation of each sample's sparse code from its class mean, alongside the usual reconstruction and linear classification error terms. To handle multiple imaging modalities, they extended SCDDL into a multi-modality version (mSCDDL) that learns a separate dictionary and classifier per modality and fuses per-modality classifier outputs through a weighted combination, with weights optimized by grid search. Using ROI-averaged features from 113 AD, 110 MCI, and 117 normal-control subjects from ADNI, mSCDDL achieved 97.4% accuracy for AD versus normal and 77.7% accuracy for MCI versus normal, outperforming each single-modality SCDDL variant as well as other established multi-modal fusion methods, while requiring substantially less computation time, demonstrating that a compact learned dictionary of just 20 atoms can rival methods that use the entire training set as an implicit dictionary.

The same research group subsequently pushed this framework into a kernelized setting to better exploit cross-modal complementarity. Li et al. introduced multi-feature kernel SCDDL (MKSCDDL), which maps each modality's features into a higher-dimensional space via Mercer kernels before applying the SCDDL objective, expressing the dictionary implicitly as a linear combination of kernel-mapped training samples. This allows the same reconstruction, classification-error, and within-class-similarity terms can be optimized without explicitly forming the high-dimensional mapping. Sub-kernels for each modality are then combined, extending the earlier

weighted-combination scheme of mSCDDL into a single unified kernel objective. Evaluated on the same three-modality ADNI cohort, MKSCDDL improved classification accuracy over mSCDDL and other multi-modal baselines across all three diagnostic contrasts: 98.2% for AD vs. cognitively unimpaired (CU), 78.5% for MCI vs. CU, and 74.5% for AD vs. MCI, while also reducing test time.

Most relevant to our study, Liyanage et al. tackle COVID-19 detection from chest CT by combining two complementary DL strategies rather than a single dictionary. The first, frozen dictionary learning, addresses class imbalance in the training data, where normal lung-field patches vastly outnumber abnormal patches by learning dictionary atoms hierarchically. The second, LC-KSVD, learns a discriminative dictionary directly from patient-level class labels (normal, common pneumonia, novel coronavirus pneumonia). The two dictionaries are concatenated into a single augmented dictionary, patches are sparse-coded against it via OMP, and simple per-scan statistics (mean, variance, non-zero count) of the sparse coefficients associated with each class-relevant dictionary segment are then fed to an SVM classifier. On the cleaned CC-CCII CT dataset, this patch-based sparse-coding pipeline achieved 89% accuracy for binary NCP-vs-other classification and 82.6% accuracy for the three-class problem, matching or slightly exceeding a benchmark ResNet3D-34 deep learning baseline on the same data. The authors emphasize that because dictionary atoms live in the same domain as the input patches, the resulting classifier offers a more interpretable link between learned features and diagnostic categories than an end-to-end deep network directly echoing the discriminative-dictionary theme seen in Dong et al. and Tong et al., but here applied to whole-scan disease classification rather than pixel-wise segmentation.

Chapter 3

Methodology

Figure 3.1 gives a visual overview of the full framework presented in this study.

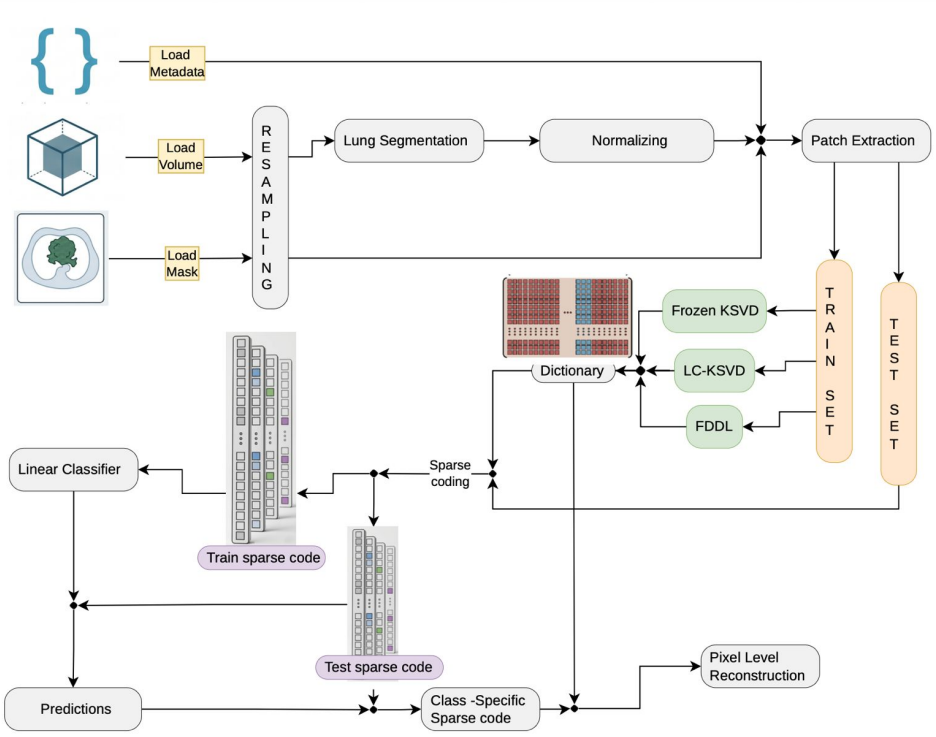

Figure 3.1: Overall Methodology Diagram.

3.1 Data Acquisition

The quality and consistency of the input data has a direct impact on how well the dictionary learning stage can later separate abnormalities from normal lung, so a fair amount of the early project time went into data acquisition and preparation before any model code was written. This section walks through the datasets explored and chosen, how the raw CT volumes were loaded and experimented on, the different lung segmentation strategies that were tried, and how the selected lung volumes were sampled into 3D patches that feed the Dictionary Learning models.

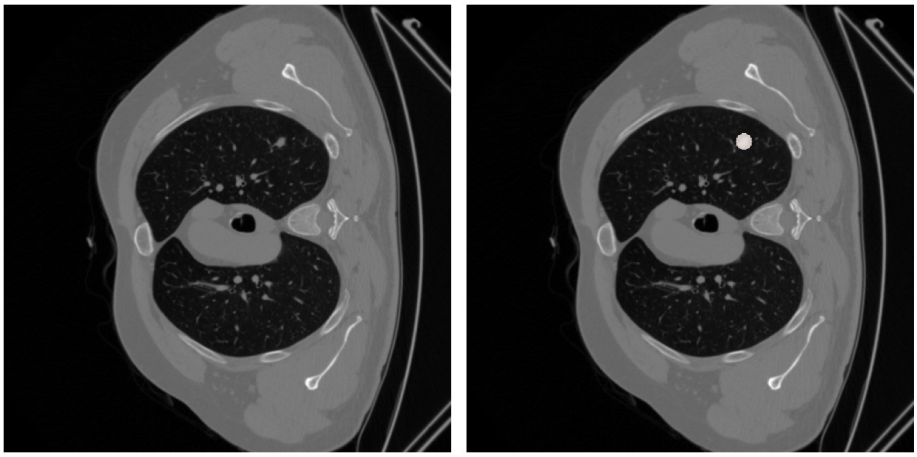

Figure 3.2: Example CT scan with ReXGroundingCT mask overlaid over a scan from CT-RATE.

3.1.1 Dataset Selection

Multi-abnormality classification and segmentation for 3D chest CT images requires a dataset that includes multiple CT abnormalities and their corresponding voxel-level annotations. Only three datasets fulfilled the former requirement:

1. CT-RATE[25]
2. RadGenome-Chest CT[60]
3. Rad-ChestCT[61]

Of these, only CT-RATE and RadGenome-Chest CT were publicly available, while Rad-ChestCT only had an initial release of 3,630 chest CT scans (approximately 10% of the dataset) and the files were restricted to the public. Rad-ChestCT was also used as the external validation set for training the CT-CLIP model, the flagship contrastive model was trained on the CT-RATE dataset [24]. RadGenome-Chest CT extends the CT-RATE dataset with with 197 category organ-level masks and 665,000 grounded text reports for vision-language AI models. However, neither of these provides the necessary spatially localized annotations to train a 3D lung segmentation model. Fortunately, ReXGroundingCT[62] was released very close to the start of this project, and it provides per-finding voxel-level segmentation masks for a subset of the CT-RATE dataset. Figure 3.2 shows an example CT scan with ReXGroundingCT mask overlaid over a scan from CT-RATE.

CT-RATE Dataset

CT-RATE [25] is a large-scale 3D chest CT dataset in NifTI format, hosted on Hugging Face that provides volume-level multi-label annotations and corresponding textual radiology reports across 18 distinct abnormality categories. The dataset comprises 25,692 non-contrast 3D chest CT scans from 21,304 unique male and female patients expanded to 50,188 total volumes via different reconstructions. It is split into a training cohort of 47,149 volumes from 20,000 patients and a validation cohort of 3,039 volumes from 1,304 patients. Table 3.1 shows the 18 abnormality categories in CT-RATE.

Medical material	Lung nodule
Arterial wall calcification	Lung opacity
Cardiomegaly	Pulmonary fibrotic sequela
Pericardial effusion	Pleural effusion
Coronary artery wall calcification	Mosaic attenuation pattern
Hiatal hernia	Peribronchial thickening
Lymphadenopathy	Consolidation
Emphysema	Bronchiectasis
Atelectasis	Interlobular septal thickening

Table 3.1: List of abnormalities in the CT-RATE dataset.

CT-RATE captures substantial heterogeneity in CT scanners, acquisition protocols, and reconstruction techniques. It comprises scans from four primary scanner manufacturers:

1. Siemens Healthineers
2. SIEMENS
3. Philips Healthcare
4. Philips-Neusoft Medical Systems

ReXGrounding-CT Dataset

ReXGroundingCT, is the first publicly available dataset linking voxel-level 3D segmentation masks to free-text findings in chest CT scans in NifTI fromat [62], hosted on Hugging Face. It includes 3,142 non-contrast chest CT studies derived from the CT-RATE dataset organised as a train set of 2,992 scans, a valid set of 50 scans and a held-out private test set of 100 studies. This test set is hosted on RexRank [63], a public leaderboard and challenge for assessing AI-powered radiology report generation. ReXGroundingCT enables grounding for 14 abnormality categories where 79% of the annotated findings represent focal abnormalities such as nodules or localized consolidations, while 21% represent diffuse or infiltrative patterns. Table 3.2 shows the list of abnormalities in the ReXGroundingCT dataset, divided into typically focal and typically non-focal categories.

Table 3.2: List of Abnormalities in the ReXGroundingCT Dataset

Typically Focal	**Typically Non-Focal**
Pulmonary nodules and masses	Emphysema (including Centrilobular,
Consolidation / atelectasis	Paraseptal, Bullous)
Ground-glass opacities	Bronchiectasis
Pleural effusion / thickening	Bronchial wall thickening
Linear opacities (including subsegmental	Septal thickening (including Interlobular,
atelectasis, scarring, fibrosis)	Reticulation)
Honeycombing	Micronodules (including Centrilobular,
Pneumothorax	Tree-in-bud, Perilymphatic)
Other focal findings	Other non-focal findings

A segmentation mask is 4-Dimensional ($F \times H \times W \times D$) where F is the number of findings. Scans feature HxW resolutions primarily at 512×512 pixels with slice counts spanning 104 to 1,005 slices. Background Pixels are represented by a value of 0. For any given channel along the F dimension, each unique non-zero integer represents a distinct entity instance belonging to that finding.

3.1.2 Target Abnormality Selection

The target abnormality classes were selected based on several factors:

- Expert Radiologist Recommendation
- Sufficient Representation in the Dataset
- Clinical Significance

To obtain an expert recommendation from a radiologist, a consultation session was held with a Consultant Radiologist at the Teaching Hospital, Karapitiya to discuss the available findings in the ReXGroundingCT datasets. Figure 3.3 shows a group photo taken after the consultation session.

Figure 3.3: First consultation session with the Consultant Radiologist

The radiologist was shown the available finding categories in ReXGroundingCT and asked which abnormalities would be most valuable to detect automatically, and which were common enough in practice to justify the modelling effort. The radiologist highlighted the following findings as having a well-established radiological basis for early intervention, making them good candidates for a proof-of-concept CAD system:

- Ground-glass opacity
- Pulmonary nodules/masses
- Consolidation
- Bronchiectasis
- Emphysema

Table 3.3: Pulmonary abnormality taxonomy in OmniAbnorm-CT.

Category	**Abnormalities**
Pulmonary parenchyma	Atelectasis, Incomplete lung expansion, Consolidation, Ground-glass opacities, Emphysema, Solitary nodule or mass, Diffusely distributed multiple nodules, Parenchymal fibrosis, Thin-walled cavitation; Thick-walled cavities; Cystic mass.
Bronchi	Bronchiectasis, Bronchial wall thickening, stenosis, or occlusion, Bronchial foreign body.
Lung interstitial	Lung interstitial fibrosis and thickening; Honeycomb lung.
Pleura	Pleural thickening; Pleural effusion; Pneumothorax; Pleural calcification; Pleural soft tissue mass.

The radiologist provided valuable insights into the clinical significance of each abnormality, their typical appearance in CT scans, and the challenges associated with their detection. For the initial phase of our study, we decided to narrow down the target abnormalities further, as the radiologist's recommendations were quite broad.

We also observed discrepancies between the abnormality classes in the CT-RATE and ReXGrounding-CT datasets. Firstly, CT-RATE does not have a class for GGO explicitly. Instead, it has a broader class called "Opacities", which contains various generalized density increases, non-specific haziness, and explicit GGOs grouped together under this broader class. This is offset by the fact that ReXGrounding-CT treats GGO as an independent, dedicated focal category.

Secondly, ReXGrounding-CT groups Atelectasis and Consolidation as one finding (see Table 3.2). This could be because they share nearly identical visual appearances and overlapping imaging features on chest CT scans [64]. Both conditions cause increased density and whitening of the lung parenchyma that obscures underlying blood vessels, and frequently present with visible air bronchograms within the affected area. This is further confirmed by the ReXGrounding-CT free-text reports, which describe many findings as "consolidation or atelectasis".

Their categorization pipeline and schema were constructed via LLM extraction and verified by board-certified radiologists [62], which suggests that the grouping was a deliberate decision made by the authors. However, another study [65] provides a comprehensive hierarchical taxonomy of CT abnormalities, with 404 representative abnormal findings across all body regions. This study separates Atelectases and Consolidation as distinct categories. Table 3.3 shows the taxonomy of the "Lung" category.

This taxonomy was developed by 7 senior radiologists with at least 10-year experience [65]. Through consultation with our own radiologist, it was confirmed that Atelectasis and Consolidation should be distinct categories as they represent different pathological processes as noted in Section 2.1.2. Because there are practical constraints in dividing the already-grouped Atelectasis/Consolidation category in ReXGrounding-CT, we decided consider it as a limitation and remove them from the initial phase of the study. To further reduce the target abnormalities, we considered the representation of each class in our dataset which is discussed in Section 3.1.3. We

finally decided on 2 target abnormalities for the initial phase of the study:

1. GGO
2. Nodules/masses

3.1.3 Exploratory Data Analysis

Based on the radiologists feedback, we performed an exploratory data analysis (EDA) and cross-checked against how well each candidate class was actually represented in the annotated data. Table 3.4 shows the distribution of the 14 finding categories in ReXGroundingCT.

Table 3.4: Distribution of findings and scans across ReXGroundingCT Categories. Categories starting with 1 are typically non-focal, while those starting with 2 are typically focal.

Category	# Scans	# Findings	Description
1a	213	237	Bronchial wall thickening
1b	249	285	Bronchiectasis
1c	421	449	Emphysema (including centrilobular, paraseptal, bullous)
1d	188	198	Septal thickening (including interlobular septal thickening, reticulation)
1e	246	321	Micronodules (including centrilobular, tree-in-bud, perilymphatic)
1f	145	153	Other non-focal abnormalities
2a	815	1135	Linear (including subsegmental atelectasis, scarring, fibrosis)
2b	924	1384	Atelectasis, consolidation
2c	1144	1527	Groundglass opacity
2d	1333	1778	Pulmonary nodules/masses
2e	186	241	Pleural effusion or thickening
2f	15	16	Honeycombing
2g	15	18	Pneumothorax
2h	58	60	Other focal abnormalities

Based on the number of findings, it was clear that GGO, Pulmonary Nodules/-masses and Atelectasis/Consolidation were the most well-represented categories in the dataset, with 1,527, 1,778 and 1,384 findings respectively. Since we decided to remove the grouped Atelectasis/Consolidation category, we were left with GGO and Pulmonary Nodules/masses as the two main target abnormalities.

Beyond how many findings each category contributed, it was also important to understand how large a typical finding actually is, since this has a direct impact on

Table 3.5: Voxel-count statistics of target abnormality annotations.

Category	Count	Mean	Median	Q1	Q3
1b	285	118,120	26,843	4,162	159,705
1c	449	338,505	19,594	3,633	151,685
2b	1384	114,153	22,820	6,418	84,033
2c	1,543	113,871	19,682	4,818	83,380
2d	1,801	10,196	417	194	1,167

how patches would later need to be sampled around each lesion. For each abnormality annotation in the dataset, the number of foreground voxels was obtained from the metadata. Table 3.5 summarises the resulting distributions.

While the first four categories have similar size distributions, the 2d category (nodule/mass) is significantly smaller. The 2d (nodule/mass) category shows a clear size disparity with the rest of the abnormalities which all have around 20,000 voxels, roughly 47 times larger than the median nodule/mass annotation at just 417 voxels. All categories have heavily right-skewed size profiles with long tails of much larger outliers. Figure 3.4 shows a box plot of voxel distributions for both classes.

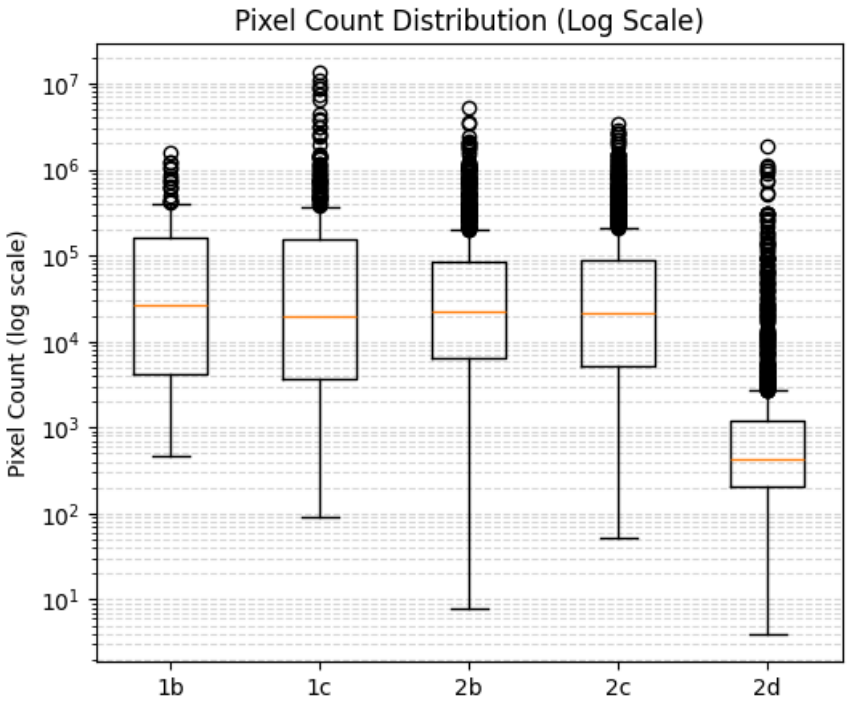

Figure 3.4: Log-scale voxel-count distribution of target abnormalities.

The size disparity of the 2d category becomes a concern when deciding on a fixed patch size for the extraction phase. Even a patch size of $16 \times 16 \times 16$ (4,096 voxels) is likely to be mostly lesion tissue for abnormalities like GGO, whereas a nodule is frequently small enough that a patch covering its bounding box will contain a substantial amount of surrounding normal parenchyma. This is somewhat offset by the use of a 50% lesion-voxel-fraction threshold during patch extraction (see Section 3.2.4), but still results in more than 75% of nodule patches being discarded, even when using a patch size is reduced to $12 \times 12 \times 12$ (1,728 voxels). This is another

Table 3.6: NIfTI header attributes for the CT volumes and corresponding segmentation masks.

Header Attribute	**CT Volume**	**Segmentation Mask**
Dimensions	[3, 512, 512, N_z, 1, 1, 1, 1]	[4, 1, 512, 512, N_z, 1, 1, 1]
Data type	int16	uint8
Bits per voxel	16	8
Voxel spacing	$[1, \Delta_x, \Delta_y, \Delta_z, 0, 0, 0, 0]$	$[1, 1, 1, 1, 0, 1, 1, 1]$
Spatial units	Millimetres	Millimetres
Intensity scaling	(s, i)	(s, i)
Affine transformation	Scanner-space affine	Identity affine
Image origin	(t_x, t_y, t_z)	$(0, 0, 0)$

limitation of our study and may discard small nodules and only include larger masses which are typically larger than 30mm.

3.1.4 CT Volume and Mask Loading

Each CT scan is stored as a 3D NIfTI volume of dimensions $512 \times 512 \times N_z$ with voxel spacing $\Delta_x, \Delta_y, \Delta_z$ mm and 16-bit signed integer intensities representing Hounsfield Units (HU). The corresponding segmentation mask is stored as an 8-bit unsigned integer 4D NIfTI volume with matching 3D spatial dimensions plus an additional singleton channel dimension containing integer label values.

Each NifTI file contains a header with metadata describing the volume's attributes. Table 3.6 lists the relevant header fields used in this study. The CT volumes preserve the scanner-space affine transformation, which maps voxel indices to physical coordinates in millimetres. In general, the affine matrix can be represented as:

$$\mathbf{A} = \begin{bmatrix} -\Delta_x & 0 & 0 & t_x \\ 0 & -\Delta_y & 0 & t_y \\ 0 & 0 & \Delta_z & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

where Δ_x, Δ_y, and Δ_z denote the voxel spacing along the x-, y-, and z-axes, respectively, and (t_x, t_y, t_z) denotes the translation of the image origin in scanner coordinates. The negative scaling factors along the x- and y-axes indicate that these axes are stored in the opposite direction relative to the scanner coordinate system, while the z-axis retains its positive orientation, which is common for DICOM-derived CT scans.

Although each segmentation mask corresponds exactly with its associated CT scan, the mask NIfTI headers do not preserve the original spatial metadata. Specifically, the masks are stored with unit voxel spacing ($1 \times 1 \times 1$ mm), and an identity affine transformation, whereas the CT volumes retain their scanner-space affine and true voxel spacing. During preprocessing, the mask metadata is updated to match the corresponding CT volume's metadata.

Volumes were loaded using the nibabel Python library and converted into float32 HU arrays, with each scan represented as:

$$I(x, y, z) \in \mathbb{R}^{H \times W \times D}$$

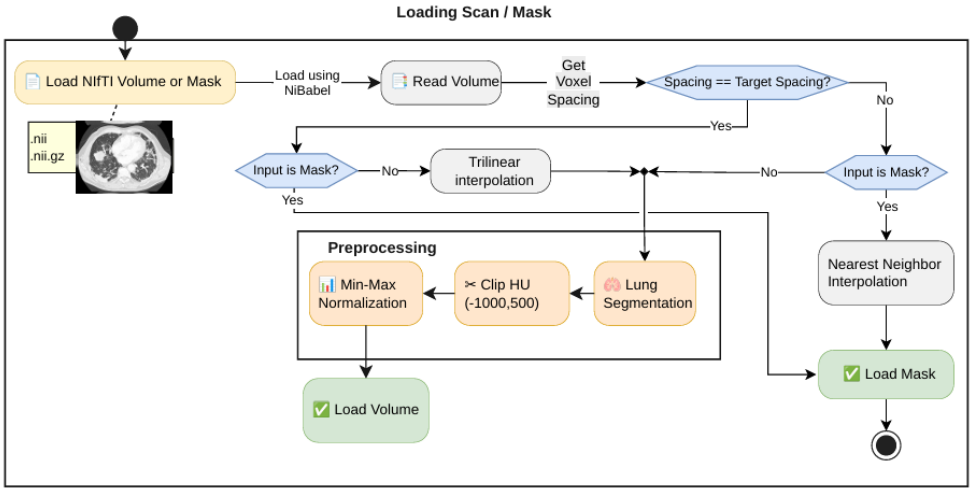

Figure 3.5: Shared loading pipeline for CT volumes and finding masks

where H, W and D denote the height, width and depth of the volume.

3.2 Data Preprocessing

Figure 3.5 summarises the overall data pre-processing pipeline used in this study. It consists of a shared loading pipeline for both CT volumes and their corresponding segmentation masks, followed by resampling to a common voxel spacing, lung segmentation, and intensity normalization.

3.2.1 Resampling

Due to the variation in voxel spacing across the CT-RATE dataset, all volumes had to be resampled to a common voxel spacing to correct for variable scanner resolutions. We chose the same anisotropic target spacing used by the pre-processing pipeline of the CT-CLIP model, since it was trained on the volumes from the same dataset as ours. The target spacing was set to $0.75 \times 0.75 \times 1.5$ mm, in the x, y and z directions respectively.

An in-plane (X-Y) resolution of 0.75 mm preserves relatively fine spatial detail and limits the loss of small structures such as certain pulmonary nodules due to excessive downsampling. Furthermore, chest CTs naturally often have higher in-plane resolution than axial slice thickness (Z). Setting a 1.5 mm Z-spacing respects this clinical reality while reduces the number of voxels by approximately 50% compared to isotropic 0.75 mm 3D grids:

$$V_{\text{voxel, iso}} = 0.75 \times 0.75 \times 0.75 = 0.421875 \text{ mm}^3,$$

$$V_{\text{voxel, anisotropic}} = 0.75 \times 0.75 \times 1.5 = 0.84375 \text{ mm}^3.$$

$$\frac{N_{\text{anisotropic}}}{N_{\text{iso}}} = \frac{V_{\text{voxel, iso}}}{V_{\text{voxel, anisotropic}}} = \frac{0.421875}{0.84375} = 0.5.$$

Figure 3.6: Distribution of voxel spacings across the dataset

The resampling was performed using trilinear interpolation for the CT volumes and nearest-neighbour interpolation for the segmentation masks. The zoom factors were calculated as:

$$\text{zoom}_x = \frac{\Delta_x}{0.75}, \quad \text{zoom}_y = \frac{\Delta_y}{0.75}, \quad \text{zoom}_z = \frac{\Delta_z}{1.5} \tag{3.1}$$

This was further confirmed by analysing the distribution of voxel spacings across the dataset. Figure 3.6 shows this distribution along each axis.

After resampling, all volumes and masks were resized to a fixed matrix size of $480 \times 480 \times 240$ voxels across the coronal, sagittal and axial axes respectively, using center-cropping for oversized volumes or padding for undersized volumes. This follows the same approach as the CT-CLIP model, yielding a field of view of $360 \times 360 \times 360$ mm, which spans the physical volume of a human thorax, ensuring that critical boundaries are not cut off during cropping.

3.2.2 Lung Segmentation

Passing the whole volume to the model would include information on bones, soft tissue and background air which adds computational overhead without contributing anything useful to the models learning process, or worse, risks letting the model pick up spurious patterns unrelated to lung pathology. Therefore, before any patches could be extracted, the raw volumes had to be reduced to just the lung region.

We explored three different approaches to isolate the lungs.

Classical Image Processing

The first attempt used a fairly standard HU-threshold pipeline, since lung tissue is normally well separated from surrounding structures in intensity. Each volume was first thresholded to keep only voxels between -900 and -200 HU, the range typically associated with aerated lung parenchyma:

$$M_{thresh}(x, y, z) = \begin{cases} 1, & -900 \leq I(x, y, z) \leq -200 \\ 0, & \text{otherwise} \end{cases} \tag{3.2}$$

This crude threshold mask is noisy on its own, so it was cleaned up with a short sequence of morphological steps applied slice-by-slice and volume-wise. Border-connected regions were removed on each axial slice, mainly to drop the surrounding air and imaging-table artefacts that also fall inside the HU window. Afterwards, only the two largest connected components were kept under the assumption that these correspond to the left and right lung. A 3D binary closing (a $3 \times 3 \times 3$ connectivity structure with 2 iterations) was then used to smooth the boundary and reconnect thin gaps, followed by binary hole-filling to plug any internal cavities left inside the mask.

On visibly normal scans, this five-stage pipeline produced reasonable lung masks when checked manually. However, Consolidation and areas of lung opacity raise the local tissue density towards that of soft tissue, so the affected voxels are not

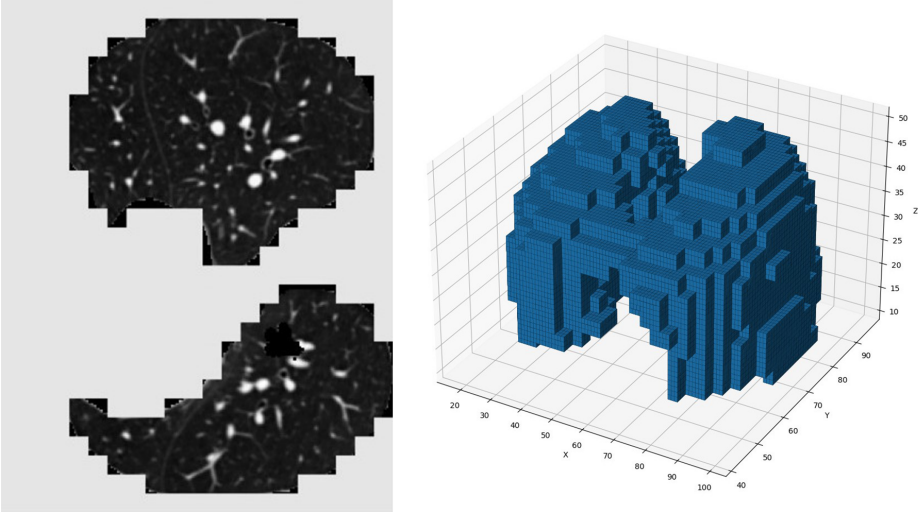

Figure 3.7: LungMask segmentation result on a scan with GGO and consolidation.

registered as lung parenchyma under the initial −900 to −200 HU window. Since the lung segmentation fails at the very first thresholding step, we were motivated to move away from purely intensity-based segmentation and towards a learned segmentation model.

Deep Learning-based Segmentation

First, TotalSegmentator [16] was considered for lung segmentation. Although it can segment the lungs, lung segmentation is only one part of its broader whole-body segmentation task. In the "total" task, the lungs are represented as five separate lobe classes: left upper lobe, left lower lobe, right upper lobe, right middle lobe, and right lower lobe. These five masks therefore need to be combined to obtain a single whole-lung mask.

This additional merging step makes the preprocessing pipeline more complex and may introduce unwanted boundaries between adjacent lobes. Since the proposed pipeline requires a single lung mask for the subsequent processing steps, this representation was less suitable for the project. Therefore, TotalSegmentator was not selected for the final pipeline.

The method that was ultimately adopted is the LungMask library, which uses a U-net (R231) model pretrained on diverse CT scan data for general lung mask generation. Unlike the classical pipeline, LungMask is a trained segmentation network built specifically for whole-lung extraction, and it proved noticeably more stable when abnormal tissue was present. Consolidation and GGO regions inside the lung boundary appeared to retain on manual inspection. Figure 3.7 shows a LungMask segmentation result on a scan with GGO and consolidation.

Given a CT volume, LungMask returns a label map with three values: 0 for background, 1 for the left lung and 2 for the right lung. This was converted into a binary mask and applied to the original volume. Voxels falling outside the mask were reset to a fixed background value of −1000 HU (approximating air).

LungMask is built on top of SimpleITK, which stores volumes in (z, y, x) order rather than the standard (x, y, z) layout. So, each resampled numpy array was transposed accordingly before being ingesting the image. The resulting image object was

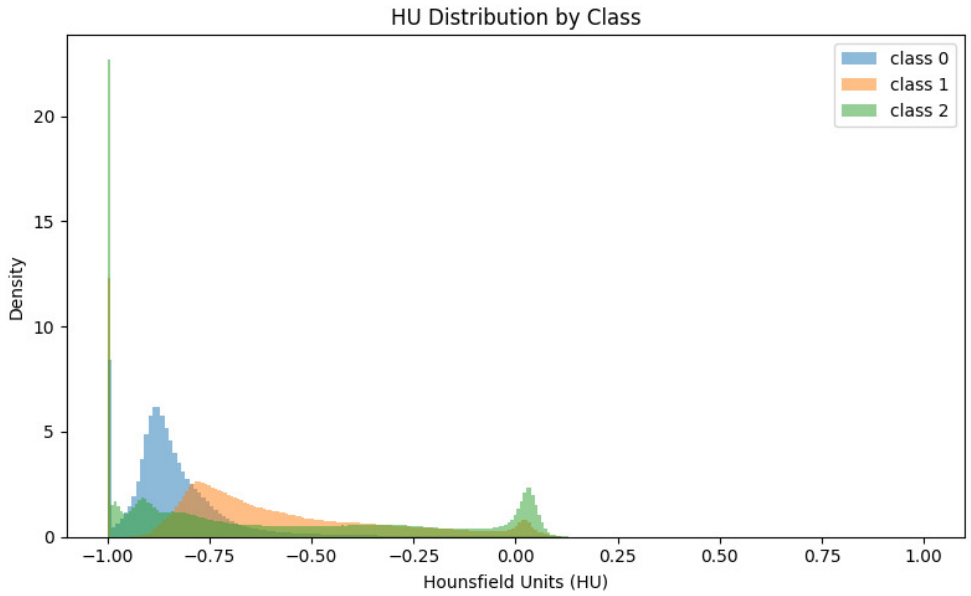

Figure 3.8: Distribution of HU values across the dataset inside the abnormality masks.

then resampled with the same spacing used in section 3.2.1, Then, it was passed to the LungMask inferer described above, with the resulting label map transposed back to the original (x, y, z) ordering before the mask was applied.

3.2.3 Normalization

After lung segmentation, the voxel intensities were clipped to a fixed window of $[-1000, 200]$ HU. $-1,000$ HU corresponds to air, while the 200 HU upper limit was decided using a histogram analysis of the dataset. Figure 3.8 shows the distribution of HU values across the dataset inside the abnormality masks. For Normalization, we considered fixed Min-Max scaling to $[-1, 1]$ following the same approach used in the CT-CLIP study [24].

3.2.4 Patch Extraction

Feeding entire volumes directly into the dictionary learning stage is not practical as a single scan can contain tens of millions of voxels. Therefore, the final step was to break each volume down into smaller 3D patches. This patch-based sampling provides the following benefits:

- Allows the model to focus on local structural patterns rather than whole-organ context.
- Reduces memory requirements by limiting the number of voxels processed at once.
- Since the proposed dictionary learning models are linear, the abnormal patches can be mapped back to the original volume.

There are 3 competing forces influencing the patch size selection:

Patch Size	# voxels	50% of Voxels
$32 \times 32 \times 16$	16,384	8,192
$16 \times 16 \times 8$	2,048	1,024
$12 \times 12 \times 6$	864	432

Table 3.7: Comparison of candidate patch sizes.

1. Size of Nodules – Patch size must be lowered to capture smaller nodules.
2. Size of Dictionary – Patch size must be lowered to keep the dictionary size manageable.
3. Loss of context – Patch size must be increased to capture more context around the abnormality.
4. Dataset variability – Smaller patch sizes allow for more patches to be sampled from the same volume.

Furthermore, since abnormalities are irregularly shaped, a maximum of 50% of the voxels in a patch were allowed to be non-lesion. A similar approach was used for healthy tissue, where a maximum of 50% of the voxels in a patch were allowed to be background to avoid patches sampled completely from outside the lung parenchyma.

At 50% of the median nodule size, we considered multiple patch sizes that would include a sufficient number of patches from the nodule/mass category. This distribution is shown in table 3.7. Since th Z-axis spacing is double that of the X and Y axes, the patch size along the Z-axis was halved to maintain a cubic patch volume. The influence of the size of the dictionary is discussed in Section 3.3. The loss of context stems from the fact that smaller patch sizes may be unable to capture the surrounding tissue context of the abnormality, which is important for classification. This is something that is not easily quantifiable, and is a limitation of our study. The final tradeoff is dataset variability, where smaller patch sizes allow for more patches to be sampled from the same volume. This reduces the ability of the model to generalize to unseen data, and is also a limitation of our study.

Based on the above tradeoffs, a patch size of $12 \times 12 \times 6$ was chosen for this study. At the resampled spacing, this corresponds to roughly a $9 \times 9 \times 9\text{mm}^3$ region of lung tissue, which is large enough to capture a typical nodule or a local patch of opacity. Patches were sampled differently depending on whether they were meant to represent normal tissue or a target abnormality. Figure 3.9 lays this out visually.

- **Normal patches** were drawn on a non-overlapping grid across the entire lung volume, with a stride equal to the patch size. A patch was discarded if more than half of its voxels were near-zero after masking, since that generally indicates the patch sits mostly outside the lung region rather than within actual tissue.
- **Abnormal patches** were sampled only from scans carrying a ground-truth finding mask. Since a single scan can have several individual findings that belong to the same target category, the relevant finding channels of the 4D mask were first OR-combined into one binary lesion mask per category present in that scan. A tight axis-aligned bounding box was then computed around the foreground of that lesion mask, and the patch window was slid across the

box with a stride of 4 voxels in each direction, making it markedly denser than the normal-patch grid, since the region of interest here is already small and every overlapping view of it is potentially useful. Each candidate patch was only kept if the fraction of lesion-labelled voxels within its corresponding mask region reached a threshold of 50%.

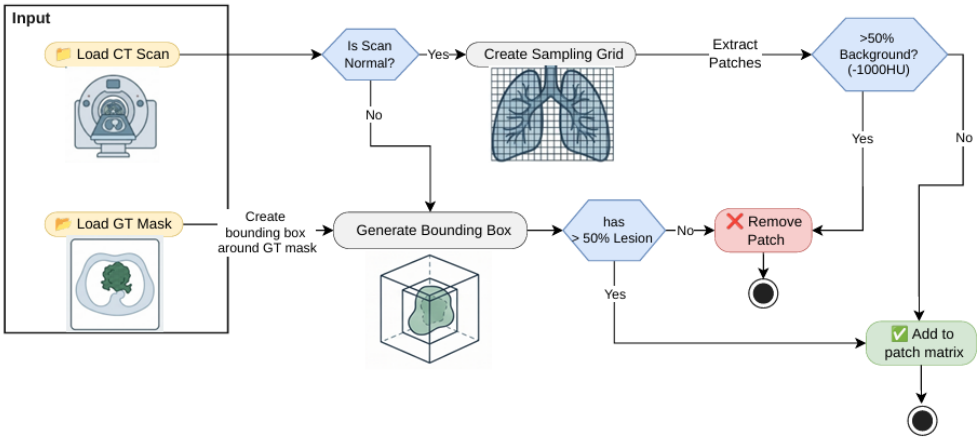

Figure 3.9: Patch extraction workflow.

Each extracted patch is subsequently flattened into a vector representation before being handed to the dictionary learning algorithms. Patches extracted this way were saved per scan and per class as compressed archives containing the flattened feature matrix, class labels, the originating scan identifiers and the voxel coordinates of each patch, and were later concatenated across classes to build the full training matrix used for the dictionary learning algorithms.

3.3 Dictionary Learning

Once class-wise patches had been extracted and flattened into vectors, the next stage was to learn a dictionary that could represent these patches as a sparse combination of a small number of atoms. Four dictionary learning algorithms were implemented and compared for this purpose: K-SVD, LC-KSVD, FDDL, and Frozen Dictionary Learning. All of the dictionary learning code lives in a separate open-source PyPI package built specifically for this project. The code can be found on Github. All four algorithms take the same flattened patch matrix and produce a dictionary D that patches can later be sparse-coded against.

3.3.1 Deriving Practical Baselines for s, N, and K

Before training any of the dictionary learning algorithms, it is important to understand the theoretical limits on the sparsity level s, the number of training samples N, and the dictionary size K. These limits are derived from the properties of the dictionary and the underlying sparse coding problem. The uniqueness of sparse representations is governed by the dictionary's *mutual coherence*, which is the maximum absolute inner product between any pair of normalized columns. This provides a practical condition for the sparsity level:

$$s < \frac{1}{2}\left(1 + \frac{1}{\mu(D)}\right),$$

where:

$$\mu(D) \geq \sqrt{\frac{K-d}{d(K-1)}}.$$

This is known as the Welch Bound [66]. As K increases, the bound approaches $(\mu(D) \approx 1/\sqrt{d})$, yielding the asymptotic sparsity ceiling:

$$s \lesssim \frac{\sqrt{d}}{2},$$

which is independent of K. For d=864, this gives $s_{\max} \approx 14$. From this, we decided that the sparsity level of s=10 is well within the uniqueness region.

The original K-SVD work reports stable atom recovery when each atom is supported by approximately 100-300 training samples [46]. This gives a range of values for the dictionary size K:

$$K \lesssim \frac{100,000}{100 \text{ to } 300} \approx 333 \text{ to } 1,000$$

This is considerably smaller than the typical 2-4x overcomplete dictionaries used in image processing. Even at $K = 1,000$, $K/d \approx 1.2$, which is barely overcomplete. To achieve desirable overcompleteness, we would have to tradeoff the number of training samples per atom N/K, which could lead to unstable atom recovery. However, many published dictionary-learning results train with 20-50 samples/atom successfully, especially with mini-batch/online updates like Mairal et al.'s ODL [67], which distributes this effect over many updates, better than batch K-SVD. Table 3.8 shows the N/K values for different levels of overcompleteness.

Table 3.8: Trade-off between dictionary overcompleteness and training samples per atom for $N = 100{,}000$ and $d = 864$.

Dictionary Size (K)	**Overcompleteness (K/d)**	**Samples per Atom (N/K)**
1,728	2.0×	58
2,592	3.0×	39
3,456	4.0×	29

At $K = 3d$, the dictionary has enough overcompleteness for s=10 sparse coding to be meaningful, while keeping N/K in a range (20-50). Therefore, we decided to use $K = 3d = 2{,}592$ as the target dictionary size for all algorithms in this project.

3.3.2 Patch Normalization

A major consideration in our methodology was whether we should normalize the patches before passing them to the dictionary learning algorithms. This was motivated by the approach used in the original K-SVD image denoising paper by Elad and Aharon [51], where each patch has its local mean subtracted before dictionary learning, and the mean-subtracted patch is then divided by its own L_2 norm before being passed to OMP. This is so that the SVD step used to update dictionary atoms

is not dominated by a constant background offset, and the L_2 norm is removed so that every patch contributes to the dictionary update on a comparable scale, no matter how much contrast it originally had.

However, this approach isn't applicable, atleast directly, because of what the HU value represents physically in a CT scan. Unlike a conventional image intensity, the HU is a direct physical measure of tissue density. The raw intensity therefore carries diagnostic information on which the classification stages later depend on. With the L_2 norm applied, findings that are structurally similar but differ in density could consequently be mapped close together in the normalised space, even though this density difference is often the distinguishing factor between the two classes.

However, sending the raw patches has its own drawbacks. Consider a patch where $N = 12 \times 12 \times 12 = 1728$. The energy of a signal is proportional to the squared ℓ_2 norm:

$$E = \|\mathbf{x}\|_2^2 = \sum_{i=1}^{N} x_i^2.$$

For a typical normal lung patch, the normalized voxel values are approximately constant at -0.85. Thus,

$$\|\mathbf{x}_{\text{DC}}\|_2 = \sqrt{N(0.85)^2} = 0.85\sqrt{1728} \approx 35,$$

giving an energy of

$$E_{\text{DC}} \approx 35^2 = 1225.$$

Let's assume that the remaining texture variations have a normalized magnitude of approximately 0.05–0.1, giving

$$\|\mathbf{x}_{\text{tex}}\|_2 \approx (0.05\text{–}0.1)\sqrt{1728} \approx 4,$$

and

$$E_{\text{tex}} \approx 4^2 = 16.$$

Therefore, the Direct Current (DC) component accounts for approximately

$$\frac{E_{\text{DC}}}{E_{\text{DC}} + E_{\text{tex}}} = \frac{1225}{1225 + 16} \approx 98.7\%$$

of the total signal energy, meaning that the HU intensity dominates the sparse coding process as a feature.

We also tested an alternative approach that combines the benefits of both methods by explicitly separating intensity and texture information before sparse coding. Instead of removing the patch mean entirely or encoding it implicitly through a DC atom, each patch $\mathbf{x}$ is represented as

$$\mathbf{x} \to (\mu,\ \|\mathbf{x} - \mu\|_2,\ \boldsymbol{\alpha}_{\text{tex}}),$$

where μ is the mean attenuation, $\|\mathbf{x} - \mu\|_2$ represents the texture magnitude, and $\boldsymbol{\alpha}_{\text{tex}}$ is the sparse code of the normalized zero-mean texture,

$$\tilde{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\|\mathbf{x} - \mu\|_2}.$$

The classifier is then trained using the combined feature vector

$$[\mu,\ \|\mathbf{x} - \mu\|_2,\ \boldsymbol{\alpha}_{\text{tex}}]\,.$$

This preserves the highly interpretable mean attenuation as an explicit scalar feature while allowing the dictionary to learn only texture, preventing the sparse coding process from being dominated by the DC component.

We tested these 2 approaches using a linear SVM, trained by sending patches in 3 different ways:

1. Raw patch
2. Mean and Standard Deviation of the patch intensities
3. Mean-subtracted L2 normalized patch

The results of this experiment are discussed in Section 3.6, and showed that the second approach (using the mean and standard deviation) performs best.

3.3.3 K-SVD

K-SVD is used in our pipeline for two purposes: as a standalone baseline dictionary trained separately over all three classes, and as the underlying atom-update algorithm that both LC-KSVD and Frozen Dictionary Learning are built on. Sparse coding during training uses the Batch-OMP algorithm.

As a standalone baseline, K-SVD is trained on all classes separately and joined together into a single dictionary. We set a mutual coherence threshold of 0.99 for atom replacement, and train for 10 iteratins.

3.3.4 LC-KSVD

The LC-KSVD2 variant (Section 2.7.1) is used, so that a linear classifier is learned jointly with the dictionary. However, this

weight $\alpha = 4.0$ and classifier weight $\beta = 2.0$ (matching the settings Jiang et al. report for their face recognition experiments), over 10 outer iterations with a short 2-iteration K-SVD warm start used to initialise the dictionary before the main training loop begins.

3.3.5 FDDL

FDDL's coefficient update (Section 2.7.2) combines a class-fidelity term with a Fisher-style scatter penalty, which Batch-OMP cannot solve since it is built only for plain reconstruction error. This project instead solves it with FISTA (Fast Iterative Shrinkage-Thresholding Algorithm), following Beck and Teboulle [68]: for each class in turn, with the dictionary and every other class's codes held fixed, FISTA minimises the smooth fidelity and scatter terms together with the ℓ_1 sparsity penalty, using a backtracking line search. The dictionary itself is then updated class by class as a least-squares fit against each class's own coded patches.

FDDL is trained with a dictionary of $7 \times 1{,}728 = 12{,}096$ atoms split across the three class sub-dictionaries, sparsity weight $\lambda_1 = 0.005$, scatter weight $\lambda_2 = 0.005$, elastic term $\eta = 1.0$, over 10 outer iterations, with up to 100 FISTA iterations allowed for each class's coefficient update within an outer iteration.

3.3.6 Frozen Dictionary Learning

Frozen Dictionary Learning (Section 2.7.3) is the primary method used in this project to produce the dictionaries evaluated later in this chapter. Normal lung tissue makes up the large majority of voxels in any CT scan, while GGO and nodule patches are comparatively rare, so training a single K-SVD dictionary jointly over all three classes risks having the dictionary's atoms dominated by normal-tissue patterns. Frozen Dictionary Learning avoids this by learning the normal-tissue representation first, then explicitly reserving new atoms for each abnormality on top of it, so every class is guaranteed a dedicated part of the dictionary rather than having to compete for space with normal patches.

Training proceeds in two stages, sized specifically for this project's class distribution:

1. **Base stage.** A K-SVD dictionary is trained using only the normal patches, producing a base dictionary D_{base} of $4 \times 1{,}728 = 6{,}912$ atoms. These atoms are then frozen for every stage that follows.

2. **Residual stages.** For each abnormality class in turn (GGO, then nodule), a new K-SVD dictionary is trained on that class's patches only, with every atom learned so far passed in as frozen. Every patch is still sparse-coded jointly over the combined frozen-plus-new dictionary at each iteration, but only the new atoms are ever updated. Once a class's stage finishes, its new atoms are appended to the dictionary and frozen in turn for the next class. GGO is given a residual of $2 \times 1{,}728 = 3{,}456$ new atoms and nodule a residual of 1,728 new atoms, reflecting how much less training data is available for nodules than for GGO.

Every stage uses the same sparsity level ($T = 10$), mutual coherence threshold (0.99) and iteration count (10) as plain K-SVD, so the only difference from the standalone K-SVD baseline is this staged, frozen-atom training procedure. The result is a single combined dictionary $D = [D_{\text{base}} \mid D_{\text{GGO}} \mid D_{\text{nodule}}]$ in which every atom's class of origin is known and recorded, even though no atom was ever told this label during its own training. This bookkeeping is later used when inspecting which atoms are used for which class, as described in Section **??**'s discussion of atom activation.

3.4 Sparse Code Classification

After training the dictionaries, the patches in the training set are re-encoded against each dictionary to produce a sparse code $\boldsymbol{\alpha}^{(i)}$ for each patch i. Then, we send each code along with its associated label to a classifier, to determine the class of the patch based on its sparse representation. The classifier is trained to map the sparse codes to their corresponding class labels, allowing it to learn the relationship between the learned dictionary atoms and the target abnormalities. We trained three classifiers for each dictionary learning algorithm:

1. Logistic Regression

2. Linear SVM

3. XGBoost

The performance of each classifier was evaluated on the same dataset, and the best-performing classifier was selected for each algorithm.

3.5 Projecting back to voxel space

This is the most important part of our study as it allows us to find the specific voxels in a patch that belong to a particular class. To reconstruct the specific voxels in a patch that belong to a particular class, the feature importance of each atom must be found. There are 3 main methods we used for this:

1. Using the learned weights of a linear classifier.
2. Using class-specific sub-dictionaries.
3. Analyzing the sparse codes to find atoms that are class-exclusive.

In the case of LC-KSVD, the dictionary does not divide into class-specific sub-dictionaries. Instead, a linear classifier is learned over the sparse codes, with the classifier weights indicating the importance of each atom for each class. For a sparse code $\alpha \in \mathbb{R}^K$, the class scores can be expressed as

$$\mathbf{s} = \mathbf{W}\alpha, \tag{3.3}$$

where $\mathbf{W} \in \mathbb{R}^{C\times K}$ is the learned classifier weight matrix, C is the number of classes, and K is the number of dictionary atoms. Thus, the magnitude of $W_{c,j}$ indicates the contribution of atom j to the prediction of class c.

In contrast, for K-SVD, Frozen Dictionary Learning, and FDDL, the dictionary is divided into class-specific sub-dictionaries,

$$\mathbf{D} = [\mathbf{D}_0 \mid \mathbf{D}_1 \mid \mathbf{D}_2 \mid \cdots \mid \mathbf{D}_{C-1}], \tag{3.4}$$

where $\mathbf{D}_c$ contains the atoms associated with class c. The sparse code can consequently be partitioned in the same manner,

$$\alpha = \left[\alpha_0^\top, \alpha_1^\top, \ldots, \alpha_{C-1}^\top\right]^\top. \tag{3.5}$$

Therefore, the class-specific sub-dictionaries themselves provide an interpretable representation of the features associated with each class, while a non-linear classifier can be used to classify the resulting sparse codes.

In addition, we can analyze the sparse codes to find atoms that are class-exclusive. For each atom j and class c, the mean absolute sparse coefficient is first calculated as

$$\mu_{j,c} = \frac{1}{N_c} \sum_{i:y_i=c} |\alpha_{j,i}|, \tag{3.6}$$

where $\alpha_{j,i}$ is the sparse coefficient of atom j for sample i, y_i is the class label of sample i, and N_c is the number of samples in class c. The class selectivity of atom j for class c is then measured by

Table 3.9: Linear SVM classification performance under three input representations.

Input	Class	Precision	Recall	F1
Raw patch	0	0.95	0.92	0.93
	1	0.46	0.63	0.53
	2	0.17	0.10	0.12
Mean + Std of HU	0	0.95	0.93	0.94
	1	0.50	0.60	0.55
	2	0.27	0.20	0.23
Mean-subtracted, ℓ_2-normalized patch	0	0.86	0.87	0.87
	1	0.19	0.20	0.19
	2	0.11	0.06	0.08

$$S_{j,c} = \frac{\mu_{j,c}}{\mu_{j,\neg c} + \epsilon}, \qquad \mu_{j,\neg c} = \frac{1}{N - N_c} \sum_{i:y_i \neq c} |\alpha_{j,i}|\,, \tag{3.7}$$

where N is the total number of samples, $\mu_{j,\neg c}$ is the mean activation of atom j across all other classes, and ϵ is a small constant to prevent division by zero. A high $S_{j,c}$ indicates that atom j is strongly selective to class c relative to the other classes.

3.6 Machine Learning Models

We trained a set of classical machine learning baselines on the raw 12x12x6 raw voxel patches, instead of on their sparse codes. The same three classifiers used for the sparse code classification (Section 3.4) were used here. All three classifiers used a balanced class weighting to account for class imbalance, and hyperparameters were tuned with a grid search evaluated under five-fold group cross-validation.

During hyperparameter tuning, the input patches were grouped by scan so that patches from the same scan could not appear in both the training and validation folds of a given split. Macro-averaged F1 was used as the grid search's selection metric, so that model selection was not dominated by the majority normal class.

In order to test the hypothesis described in Section 3.3.2 we also trained a linear SVM on two additional representations of the patches: the mean and standard deviation of the voxel intensities, and the mean-subtracted, ℓ_2-normalized patch. The results of these experiments are shown in Table 3.9.

The two scalars (patch mean and standard deviation), match or exceed the 864-dimensional raw patch on every class, with class 2 nearly doubling in F1 (0.12 to 0.23) despite a 432-fold reduction in dimensionality. Since a linear classifier cannot extract more linearly-separable signal from a superset of features than from the features themselves, this indicates that intensity statistics account for essentially all of the raw patch's discriminative content, and that the remaining 862 dimensions contribute noise rather than signal at the sample sizes available for classes 1 and 2.

Removing the mean and normalizing by the ℓ_2 norm isolates texture but also discards both absolute attenuation and contrast, leaving a generic linear classifier with only raw, high-dimensional pixel values and no basis abstraction. Performance can be observed to degrade further under this representation, consistent with the raw patch's

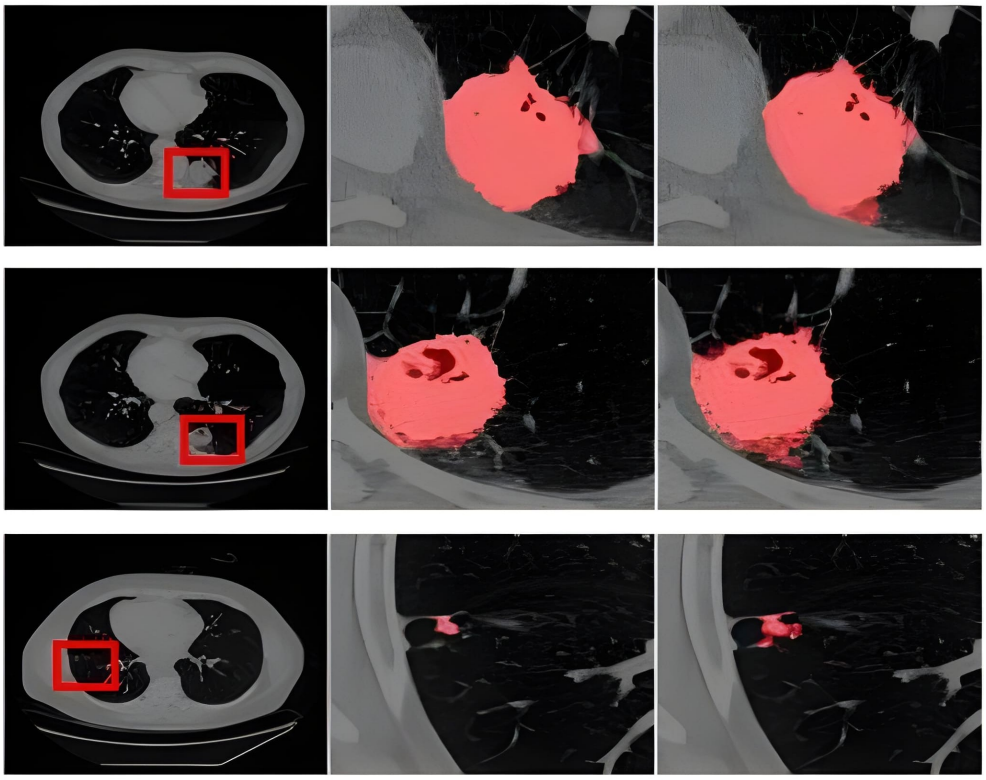

Figure 3.10: Segmentation results of nnU-Net on sample CT scans from the test set. Left column: Original scans, Middle Column: Ground Truths, Right Column: Predicted Masks.

discriminative power being concentrated almost entirely in its intensity component rather than in pixel-level texture. This result shows that texture is uninformative and the patch's intensity information is the primary driver of discriminative power.

3.7 Deep Learning Approaches

We also tested our test set on SotA deep learning models, including nnU-Net, Biomed-Parse, Merlin and CT-CLIP. The training and inference procedures for these models are described below.

3.7.1 nnU-Net

nnU-Net was the only deep learning model we explicitly trained on our dataset. This is because, nnU-Net is a self-configuring framework that automatically adapts its entire deep learning pipeline. Pre-processing, network architecture, training schemes, and post-processing are all automatically adjusted without requiring manual tuning on our part. Training was performed by sending the pre-processed CT scans and their corresponding ground-truth masks instead of the extracted patches. During inference, the trained nnU-Net model generates a complete voxel-level segmentation mask. Figure 3.10 shows the segmentation results of nnU-Net on a sample CT scans from the test set.

The held-out test CT scans are then segmented.Predictions are generated by combining the available trained models through model ensembling. If multiple trained models are available, their predictions can be combined to improve the segmentation results. The predicted segmentation masks are then compared with the corresponding binary ground-truth masks used during training. For each CT scan, the predicted and ground-truth masks are compared on a voxel-by-voxel basis. From these values, several evaluation metrics are computed, including the Dice coefficient, Intersection over Union (IoU), precision, recall, and specificity.

3.7.2 Merlin

Merlin was used as a pretrained model without any additional fine-tuning. Unlike CT-CLIP and BiomedParse, Merlin does not rely on a predefined set of disease prompts for prediction. Merlin uses a zero-shot text-image matching approach to locate abnormalities. Instead, the text descriptions provided with each ReXGroundingCT scan are used directly as input. These descriptions are converted into text embeddings, while the CT volume is processed to extract image features. The similarity between the text and image features is then calculated to identify the regions in the CT scan that correspond to the described finding.

Once the pretrained weights are loaded, a forward hook is attached to the last block of its 3D image encoder. A forward hook is a mechanism that allows the output of an intermediate neural network layer to be captured during the model's normal forward pass without modifying the layer itself. In this case, the hook is used to obtain the spatial feature maps produced by layer 4 of the inflated-3D ResNet, which retain information about the spatial location of features within the CT volume.

For each volume, the image encoder is executed once to obtain two outputs: a global image embedding representing the entire scan and a dense feature grid containing local spatial features. The global embedding is saved directly, while the dense feature grid is passed through Merlin's contrastive projection head and L2-normalised for the localisation process.

Each finding text associated with the scan is kept in its original form and separately encoded using Merlin's text encoder. The generated text embeddings are also L2-normalised. A voxel-wise cosine similarity is then computed between each finding-text embedding and every spatial location in the dense image feature grid. This produces one similarity map for each finding at the encoder's original downsampled resolution.

The similarity maps are resized back to the original CT volume resolution using trilinear interpolation and normalised to the range $[-1, 1]$. These maps represent soft localisation regions showing how strongly each area matches the given finding text. They are not segmentation masks because the Merlin model was trained for image-text similarity rather than boundary prediction.

For reporting and comparison, each free-text finding is assigned to one of the four target categories using keyword matching. For example, mentions of "nodule" or "opacity" are mapped to their corresponding categories. This categorisation is only used for grouping localisation results and does not affect the model inference.

For every case, the generated finding-level localisation maps and the global image embedding are stored as compressed arrays. A JSON record is also saved containing the original finding text, the assigned category, the ground-truth category label, the

ground-truth pixel count, and the norm of the image embedding. Previously processed cases are skipped to allow the pipeline to resume, and any failed cases are recorded in a separate error file and a run-level error list without interrupting the remaining processing.

3.7.3 BiomedParse

The official released v2 checkpoint os the BiomedParse model was used. Text prompts were used to guide the model to segment the same target abnormalities.

The pre-processed volume was then passed through BiomedParse which pads and resizes each axial slice to a fixed size of 512×512. The padding information and the axis where padding was applied were stored so that the predicted masks could later be converted back to the original CT volume size.

For each of the target abnormalities, a text prompt was created by combining several disease-related descriptions. For example, a lung nodule can be described as "lung nodule", "pulmonary nodule", or "nodular lesion". Combining these descriptions allows the model to consider different ways of describing the same abnormality in a single inference step.

The model produced a predicted mask for each text description along with an object existence score indicating the confidence that the abnormality exists. The predicted masks were resized back to 512×512 resolution and processed using BiomedParse's own post-processing method with a fixed probability threshold. The masks from all related text descriptions were then combined into one final mask for each abnormality category. The merged mask was converted into a binary foreground mask and transformed back to the original CT volume coordinates using the stored padding information.

An abnormality was considered present only when two conditions were met: the object existence score was above a predefined threshold and the predicted mask contained more than a minimum number of foreground voxels. This prevents incorrect detections caused by using only the confidence score or only the predicted mask size.

3.7.4 CT-CLIP

The released CT-LiPro linear-probe checkpoint was used here. CT-CLIP requires every input volume to have the same fixed size because the model was pretrained using a specific input resolution. For each CT volume, an empty text prompt was tokenised and passed to the model together with the preprocessed CT image using. In this process, only the image features generated by the CT encoder were used. These image features were passed through the linear classification head followed by a sigmoid function to produce independent probability values for all 18 CT-RATE abnormalities at the same time.

A second inference method was also tested using CT-CLIP's original zero-shot capability without the trained linear classification head. In this approach, each abnormality was represented using two text prompts: "abnormality is present" and "abnormality is not present". The model compared the similarity between the CT volume and both prompts, and a softmax function was used to calculate the probability of the abnormality being present.

3.8 Grad-CAM for Explainability

In addition to the segmentation and classification methods, we also experimented with Grad-CAM to understand what features affect the above deep learning models to make a prediction. Grad-CAM creates a heatmap that highlights the parts of a CT scan that had the greatest influence on the model's decision. Since the original Grad-CAM implementation is meant for 2D CNNs, a couple of implementations were developed for the model to be be applied for the Merlin and CT-CLIP models:

1. For a 3D convolutional encoder
2. For a transformer encoder

3.8.1 3D Convolutional encoder

For the convolutional encoder, Grad-CAM was applied to the last convolutional layer of the 3D ResNet backbone. A forward hook was used to save the feature maps during the forward pass, while a backward hook saved the gradients during backpropagation. These feature maps and gradients were later combined to generate the Grad-CAM heatmap.

Two target scores were tested. The first was the L_2 norm of the global image embedding. This is a text-independent target that highlights the regions that contribute the most to the overall image representation, without focusing on any particular abnormality. The second target score was the cosine similarity between the projected image embedding and the text embedding of a selected abnormality description. This is the same text-image similarity used for zero-shot localisation in Merlin (see Section 3.7.2. It allows Grad-CAM to produce a heatmap for a specific abnormality instead of only highlighting generally important regions.

$$y = \frac{z_i^\top z_t}{\|z_i\|\|z_t\|}, \tag{3.8}$$

where z_i is the projected image embedding and z_t is the embedding of the corresponding text description. By giving the model a text description of a specific abnormality, Grad-CAM can highlight the regions related to that abnormality. This text-based target was used for the quantitative evaluation.

For a selected target score, the preprocessed CT volume was passed through the encoder. The resulting scalar score was then backpropagated to the hooked layer. The gradients captured from the target layer were averaged over all spatial locations to obtain one importance weight for each feature channel. These weights were then used to calculate a weighted sum of the corresponding activation maps:

$$L = \mathrm{ReLU}\left(\sum_k \alpha_k A^k\right), \qquad \alpha_k = \frac{1}{Z}\sum_{d,h,w} \frac{\partial y}{\partial A^k_{d,h,w}} \tag{3.9}$$

where A^k is the activation map of feature channel k, y is the target score being explained, and $Z = D \times H \times W$ is the total number of spatial locations in the activation map.

The final ReLU operation removes negative values, so the heatmap keeps only the regions that positively contribute to the target score. The resulting low-resolution

activation map was resized to the original CT volume resolution using trilinear interpolation. It was then normalized to the range $[0, 1]$, producing a 3D heatmap aligned with the CT scan.

Two implementation details were important for generating a valid heatmap:

1. gradient tracking had to be explicitly enabled for the input tensor before the forward pass.

2. The encoder's contrastive projection head had to be enabled.

Without gradient tracking, the model's internal gradient checkpointing prevented the backward hook from receiving correct gradients, which produced an almost flat heatmap instead of a useful explanation. The forward pass produced the same projected image embedding used for the text-similarity target. Without the contrastive projection head enabled, the model would return only the raw pooled convolutional features.

The CT volume was preprocessed in exactly the same way as during normal inference (Section 3.7.2), including the same intensity windowing and spatial downsampling. This ensured that Grad-CAM explained the same input that the model actually used for prediction.

3.8.2 Transformer encoder

For the transformer encoder, Grad-CAM was applied to an intermediate transformer block because the model does not have a final convolutional layer like a CNN. A forward hook and a backward hook were attached to this block. The forward hook saved the activation vectors produced for each input token, while the backward hook saved the corresponding gradients during backpropagation. The activations form a tensor $A \in \mathbb{R}^{N \times K}$, where N is the number of input tokens and K is the number of feature channels.

Unlike the convolutional encoder, this model does not use text-image similarity as the target score. Instead, the target score was the predicted logit of a selected abnormality class produced by a linear classification head trained on the image embeddings. This is because the transformer encoder predicts from a fixed set of abnormality classes rather than using free-text descriptions.

The transformer encoder calculates token importance in a similar way to the convolutional encoder. However, instead of averaging the gradients over spatial locations, the gradients are averaged across all tokens:

$$\alpha_k = \frac{1}{N} \sum_{i=1}^{N} \frac{\partial y}{\partial A_i^k}, \qquad T_i = \mathrm{ReLU}\left(\sum_k \alpha_k A_i^k \right) \tag{3.10}$$

This produces one importance score T_i for each token instead of a spatial activation map. Since each token represents a fixed 3D patch of the input CT volume, these token importance scores are arranged back into their original three-dimensional token grid.

The token grid is then resized to the original CT volume resolution using trilinear interpolation and normalized using the same min-max normalization method used for the convolutional encoder. This creates a 3D heatmap that can be directly compared with the convolutional encoder heatmap, even though the two encoders use different internal feature representations.

3.8.3 Quantitative evaluation

The quantitative evaluation focused on the transformer-based Grad-CAM, for which the predicted abnormality logit was used as the target score. Grad-CAM heatmaps were generated for every case in the full test set and quantitatively compared with the corresponding ground-truth lesion masks. A forward pass was first performed for every test case to obtain the classifier's predicted nodule and opacity logits. These logits were used to rank the cases according to the model's confidence for each target abnormality for descriptive analysis.

Since Grad-CAM produces a continuous heatmap rather than a binary lesion mask, four percentile thresholds (70^{th}, 80^{th}, 90^{th}, and 95^{th}) were evaluated at the native heatmap resolution. The ground-truth mask was resized to the same resolution, and the threshold that achieved the highest Dice score was selected separately for each case. This per-case threshold selection was used as an oracle evaluation to assess the maximum spatial agreement achievable by the continuous Grad-CAM heatmap, rather than as a fixed threshold for deployment.

The selected threshold was then applied to the heatmap, and the resulting binary explanation map was upsampled to the original CT resolution. The final explanation map was evaluated against the original ground-truth mask using Dice score, Intersection over Union (IoU), Attribution IoU, and energy inside the ground-truth mask. A threshold-independent pointing-game evaluation was also performed by checking whether the voxel with the maximum Grad-CAM value over the entire volume was located inside the ground-truth lesion.

The threshold used for this evaluation was only used to measure the spatial agreement between the explanation and the annotated lesion. It was separate from any prediction threshold used by the classification model.

3.9 Evaluation Framework

The ten models compared in this study were never designed to be compared. They span sparse dictionary learning, dense voxel-wise segmentation, and promptable foundation models, and what they emit at the end of a forward pass ranges from a calibrated probability to a bare class label to a folder of binary volumes. Before a single metric can be computed, all of that has to be pushed through one common representation.

The obvious alternative, a scoring script per model, was rejected early. Ten scripts drift. Small differences in how an empty mask is handled or where a threshold is applied accumulate quietly and end up flattering whichever model the script was written for. What is used instead is one shared framework built from three pieces: a common schema that every case is converted into, one set of metric definitions applied identically everywhere, and a thin per-model conversion step that absorbs the difference in output format. Adding an eleventh model later costs one conversion step and nothing else.

3.9.1 Architecture

Those three jobs map onto three components, called throughout this section the adapter, the core, and the runner.

An adapter is the per-model conversion step. Its only responsibility is to read whatever a given model actually writes to disk, whether that is a spreadsheet of scores, a directory of mask volumes, or an array of sparse codes, and restate it as a case matching the common schema. An adapter computes nothing and judges nothing. It is a translation layer, and it is correct exactly when the case it emits is a faithful restatement of what the model predicted, no more and no less.

The core sits on the far side of that boundary. It is a single model-agnostic library of metric definitions spanning classification, localization, and explainability, and it operates only on cases already in the common schema. It does not know what an adapter is, which architecture produced a case, or what a native prediction file looks like. Because it is written once and reused, every number reported in this chapter comes from the same arithmetic, so a gap between two models reflects what they predicted rather than how each happened to be scored.

The runner is what joins the two. Given a model and an evaluation axis, it selects the matching adapter, feeds the resulting cases into the corresponding core computation, and writes out summary statistics together with per-case results. Runners are the only place where the pairing of a particular model with a particular metric computation is recorded. That was a deliberate choice: keeping the pairing outside both the adapter and the core is what allows a new model to be onboarded by writing one adapter and pointing one runner at it, instead of reopening metric code that every other result already depends on.

3.9.2 Classification Metrics

Classification is scored for CT-CLIP, Merlin, K-SVD, LC-KSVD, and Frozen Dictionary Learning, alongside three classical baselines, linear SVM, logistic regression, and XGBoost, trained directly on raw un-encoded patch vectors rather than on dictionary-learned sparse codes. Those baselines are there to ask a blunt question of the sparse-coding step: if a plain classifier on raw patches matches or beats the dictionary-learning pipeline, then the coding stage is not earning its place. The three baselines and Frozen Dictionary Learning (Section 3.3.6) are scored on a local patch-level test split rather than through an adapter against ReXGroundingCT, so their numbers sit *alongside* the ReXGroundingCT-scored models rather than directly against them.

For each target category, accuracy, precision, recall, and F1 are computed at a fixed decision threshold ($\tau = 0.5$). Where a model's score is a genuine probability, three threshold-independent metrics are added: AUROC, average precision (AP), and Expected Calibration Error (ECE). AUROC is reported as not-a-number for any class with only one label present in the evaluated set, since the metric is simply undefined there. ECE divides the unit interval into ten equal-width confidence bins and compares each bin's mean predicted confidence against its observed accuracy,

$$\mathrm{ECE} = \sum_{m=1}^{10} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|, \tag{3.11}$$

with the tenth bin closed at both ends so that a perfect confidence of 1 is not silently discarded.

Not every model produces a probability. LC-KSVD's SVM head returns a hard decision with no confidence attached, and the linear SVM baseline behaves identically

for the same reason; logistic regression and XGBoost do expose a genuine class probability and are scored with the full metric set. Merlin's zero-shot text-image matching (Section 3.7.2) is used here only to sort a free-text finding into one of the target classes, which likewise returns a category label and nothing more. Manufacturing a score by casting those decisions to $\{0, 1\}$ would be dishonest, since a rank-based metric computed over a two-point score does not mean what it means over a continuous one. Every case therefore carries its score type, and AUROC, AP, and ECE are reported as not-applicable wherever the underlying output is a hard decision. The same reasoning covers a category with no positive ground-truth examples in a split: precision and recall would collapse to zero in a way indistinguishable from genuine failure, so such categories are marked not-evaluated and dropped from the macro average.

Ground truth also differs by model. CT-CLIP is scored against CT-RATE's own per-scan annotations (Table 3.1) over a local subset, while the dictionary-learning models and Merlin are scored against ReXGroundingCT's per-finding category codes, principally ground-glass opacity (2c) and nodule (2d). The two label sets are closely related but not identical, so cross-model classification numbers should be read as indicative of relative behaviour rather than as a fully controlled head-to-head test.

3.9.3 Localization Metrics

Localization is scored for BiomedParse and nnU-Net on the deep-learning side and for K-SVD, LC-KSVD, and Frozen Dictionary Learning on the dictionary-learning side. Voxel-level agreement between a predicted mask P and a ground-truth mask G is measured by the Dice coefficient and Intersection-over-Union,

$$\mathrm{Dice}(P, G) = \frac{2|P \cap G|}{|P| + |G|}, \qquad \mathrm{IoU}(P, G) = \frac{|P \cap G|}{|P \cup G|}, \tag{3.12}$$

each returned as not-a-number when both masks are empty rather than smoothed with a small additive constant, so that *found nothing* is never confused with *found something with negligible overlap*.

A single volume-wide Dice score is a poor summary once a case contains several separate findings of the same class. It cannot separate a model that localizes one finding cleanly and misses two others from a model that diffusely over-segments the same total area. Each per-class mask is therefore decomposed into its 3D connected components and predicted instances are matched against ground-truth instances directly, following the instance-level protocol of the ReXGroundingCT benchmark [62, 63], documented at `https://rexrank.ai/ReXGroundingCT/`. A predicted instance counts as a true positive when its Dice against some ground-truth instance reaches at least 0.2, matched greedily and one-to-one:

$$\mathrm{TP} = \big|\{(p, g) : \mathrm{Dice}(p, g) \geq 0.2,\ p \leftrightarrow g \text{ under greedy matching}\}\big|. \tag{3.13}$$

Instance precision and recall are corpus-level quantities, $\mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$ and $\mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$, accumulated across the whole evaluated set before dividing rather than averaged case by case, so that a handful of cases holding very few ground-truth instances cannot dominate the result.

At whole-set level the Global Hit Rate defined by ReXGroundingCT is also reported, that is, the fraction of the F evaluated findings whose Dice clears 0.10,

$$\text{HitRate} = \frac{1}{F}\sum_{i=1}^{F} \mathbb{1}[\text{Dice}_i \geq 0.10]\,, \tag{3.14}$$

together with informal sensitivity checks at Dice thresholds of 0.05 and 0.25, which form no part of the official benchmark and are reported only as context.

nnU-Net and the dictionary-learning models all emit an already-binarised mask, so a threshold sensitivity sweep, which is meaningful for BiomedParse's continuous output, would merely reproduce the same figure at every threshold. Any case arriving with a binary mask is therefore excluded from the sweep, with the count and the reason for exclusion recorded explicitly rather than left implicit in a smaller denominator.

3.9.4 Explainability Metrics

The quantitative Grad-CAM procedure of Section **??** picks, per case, whichever percentile threshold maximises Dice against the ground-truth mask. That measures how good an explanation *could* look under the most favourable possible binarisation, which is a useful upper bound but not a deployable one, since a system at inference time has no ground truth to search over. The metric layer described here fixes a threshold in advance instead, with no oracle involved. This fixed-threshold treatment is currently applied to CT-CLIP; the dictionary-learning models are covered on the explainability axis through the oracle-threshold and atom-based procedures of Section **??**, and their numbers are therefore not interchangeable with CT-CLIP's.

The attribution map is min-max normalised to $[0, 1]$, binarised at a fixed threshold (default 0.5), and its IoU against the ground-truth mask computed exactly as in Section 3.9.3. The pointing game is stricter and asks only whether the single highest-attribution voxel lands inside the ground-truth region,

$$\text{PointingGame} = \mathbb{1}\Big[\arg\max_{v} A(v) \in G\Big]\,, \tag{3.15}$$

which has the advantage of being threshold-free and the disadvantage of ignoring everything about the map except its peak. Energy inside mask uses the whole map instead: attribution values are clipped to non-negative and the share of total positive energy falling inside the ground-truth region is reported,

$$\text{EnergyInsideMask} = \frac{\sum_{v\in G} \max(A(v), 0)}{\sum_{v} \max(A(v), 0)}. \tag{3.16}$$

Grounded accuracy ties localization back to classification: restricted to correctly classified cases, it reports the fraction whose attribution map also reaches a Dice of at least 0.10, so that a model cannot post a strong score by being confidently right for the wrong reason. Misclassified cases are excluded, because a poorly localized explanation is not the interesting failure when the prediction was already wrong.

3.9.5 Model Coverage

Table 3.10 records which axes and how many of the target categories each model is evaluated on. The coverage is uneven by design, not by omission. BiomedParse carries

no classification-style output to score, being a promptable segmentation model, and so appears on localization only. Merlin sits at the opposite corner: it is a vision-language embedding model with no segmentation capability at all, and its published classification modes, phenotype classification and five-year disease prediction, are not the ones exercised here either, since this project uses its raw image-text similarity for zero-shot matching. It is therefore scored on classification alone, derived from that similarity. nnU-Net's checkpoints are single binary foreground segmenters carrying no per-case class label, so each checkpoint has to be paired with one explicit class; checkpoints exist for ground-glass opacity and nodule, matching the abnormalities this project's own dictionary-learning models target (Section 3.3), and it is the only fully supervised segmentation baseline in the comparison. The three dictionary-learning models are the only ones present on all three axes, and CT-CLIP is the only model carrying fixed-threshold explainability numbers.

Table 3.10: Model and evaluation-axis coverage. Categories are normal, ground-glass opacity, and nodule.

Model	**Classification**	**Localization**	**Explainability**	**Classes**
Linear SVM	✓	–	–	3
Logistic Regression	✓	–	–	3
XGBoost	✓	–	–	3
CT-CLIP	✓	–	✓	3
Merlin	✓	–	–	3
BiomedParse	–	✓	–	3
nnU-Net	–	✓	–	3
K-SVD	✓	✓	✓	3
LC-KSVD	✓	✓	✓	3
Frozen Dictionary Learning	✓	✓	✓	3

Explainability for the dictionary-learning models is assessed qualitatively and through the oracle-threshold procedure of Section **??**, rather than through the fixed-threshold metrics of Section 3.9.4. Extending them to the fixed-threshold definitions is left as further work.

Frozen Dictionary Learning is scored by its own patch-level classifier on a local 44,384-patch test split rather than by an adapter against ReXGroundingCT, so its classification result is not directly comparable with the ReXGroundingCT-scored rows above.

3.10 Evaluation

Chapter 4

Results

4.1 Sparse-Coding Dictionary Learning (K-SVD Family)

4.1.1 K-SVD (Unsupervised)

4.1.2 FDDL (Fisher Discrimination Dictionary Learning)

4.1.3 Frozen Dictionary Learning

Test-Set Classification Performance

The Frozen (IncrementalFrozenDictionary) model was evaluated on a 44,384-patch test split (14,150 normal / 14,884 "2c" / 15,350 "2d" patches). A linear classifier trained on the resulting sparse codes reached 64.0% patch-level accuracy (macro F1 = 0.63); the full per-class breakdown is given in Table 4.1, with the corresponding confusion matrix in Table 4.2.

Table 4.1: Frozen dictionary patch-level classification report (test split, $n = 44{,}384$).

Class	Precision	Recall	F1-score	Support
Normal	0.66	0.87	0.75	14,150
2c (GGO)	0.55	0.40	0.46	14,884
2d (Nodule)	0.68	0.67	0.67	15,350
Macro avg	0.63	0.64	0.63	44,384
Weighted avg	0.63	0.64	0.63	44,384

Accuracy = 0.640

Table 4.2: Frozen dictionary confusion matrix (rows = actual, columns = predicted).

	Normal	2c	2d
Normal	12,256	1,434	460
2c	4,512	5,894	4,478
2d	1,685	3,401	10,264

This test split has the same class composition (14,150 / 14,884 / 15,350) as one of the LC-KSVD2 evaluation runs reported in Section 4.1.4, allowing a direct, matched comparison between the two dictionary-learning strategies rather than an apples-to-oranges one. On that identical split, the Frozen dictionary outperforms the jointly trained LC-KSVD2 model on every aggregate metric, most notably recall on the clinically important "2c" (ground-glass opacity) class (0.40 vs. 0.27, Section 4.1.4). This suggests that building the dictionary incrementally – learning a stable normal-tissue base first and adding each abnormality as a residual dictionary on top – generalises to unseen scans better than training a single dictionary jointly across all three classes at once.

Sparse Dictionary Interpretability

Sparsity and Atom Utilisation

Atom Co-occurrence and Clustering

Dictionary Coherence

Visual Inspection of Atoms and Reconstructions

4.1.4 LC-KSVD2 (Joint, Single-Shot)

Test-Set Classification Performance

The jointly-trained LC-KSVD2 model was evaluated on a 69,571-patch test split (58,764 normal / 8,019 "2c" / 2,788 "2d" patches), with five classifier heads trained on the same sparse-code representation to compare against the model's own built-in discriminative classifier. Table 4.3 summarises patch-level performance across all five.

Table 4.3: LC-KSVD2 classifier-head comparison (test split, n = 69,571: 58,764 normal / 8,019 "2c" / 2,788 "2d").

Classifier	Accuracy	Macro F1	2c F1	2d F1
LinearSVC (SVM)	**0.852**	**0.69**	0.44	**0.70**
Logistic Regression	0.751	0.57	0.38	0.48
Native LC-KSVD2 (W)	0.621	0.41	0.29	0.16
HistGradientBoosting	0.689	0.51	0.35	0.36
XGBoost	0.614	0.48	0.32	0.38

The externally-trained LinearSVC clearly outperforms every other head, including LC-KSVD2's own built-in discriminative classifier W (85.2% vs. 62.1% accuracy) – the label-consistency term learned jointly with the dictionary does not transfer as well as simply fitting a fresh linear classifier on the resulting sparse codes afterwards. The non-linear heads (HistGradientBoosting, XGBoost) do not improve on this either, both trailing plain logistic regression, which suggests the sparse-code feature space is already close to linearly separable for this task and additional model capacity does not help. LinearSVC is therefore used as the reference classifier for LC-KSVD2 throughout the rest of this chapter.

Whole-Volume Localisation

4.1.5 Raw-Patch Machine Learning Baseline

The raw-patch baselines described in Section 3.6 were evaluated on the same test split used for LC-KSVD2 Table 4.4 summarises per-class performance for the three classifiers trained on the full 864-dimension raw patch.

Table 4.4: Raw-patch classifier comparison, per class for the test split

Classifier	Accuracy	Normal F1	2c F1	2d F1
Logistic Regression	0.780	0.90	0.49	0.13
LinearSVC (MaxAbsScaler)	**0.854**	**0.93**	0.53	0.12
XGBoost	0.832	0.92	**0.54**	0.31

Overall accuracy for the best raw-patch model, LinearSVC at 85.4%, is close to LC-KSVD2's sparse-code LinearSVC at 85.2% (Table 4.3) .However, accuracy on this test split is dominated by the majority normal class, so this comparison is misleading. Computing the same macro F1 used to evaluate LC-KSVD2 gives a starkly different picture: the raw-patch LinearSVC reaches a macro F1 of only 0.53 (normal 0.93, 2c 0.53, 2d 0.12), well below LC-KSVD2's sparse-code LinearSVC at 0.69. The gap is most severe on the nodule class specifically: the raw-patch LinearSVC's 2d F1 of 0.12 (precision 0.17, recall 0.10) means it fails to reliably detect nodules from raw intensities alone, compared to an F1 of 0.70 for the same classifier trained on sparse codes. The non-linear tree baselines (XGBoost) do somewhat better on the nodule class than the raw-patch linear models (2d F1 of 0.33 and 0.31 respectively), but still fall well short of the sparse-code LinearSVC.

The two-feature (mean HU, std HU) ablation reached 80.8% accuracy and a 2d F1 of 0.21 with Logistic Regression, and 86.5% accuracy with a 2d F1 of 0.23 with LinearSVC . Capturing a substantial fraction of the full raw-patch models' accuracy from coarse intensity statistics alone, but with nodule-class performance no better than the full-patch raw baselines. Taken together, these results indicate that classifying raw patch intensities, whether the full 864-dimension patch or just its two summary statistics, recovers most of the normal-vs-abnormal signal but is a substantially worse representation than the learned sparse code for distinguishing the two abnormality classes from each other, particularly the smaller and rarer nodule class consistent with the dictionary learning and sparse coding stages contributing meaningfully beyond what raw intensity alone provides.

4.2 nnU-Net Segmentation Results

4.3 CT-CLIP Results

4.3.1 Linear-Probe (LiPro) Classification Performance

4.3.2 Zero-Shot Text-Prompt Pilot

4.3.3 Explainability (Grad-CAM)

The token-based Grad-CAM variant described in Section **??**, using the predicted abnormality logit as the target score, was evaluated on the full test set of ReXGroundingCT-annotated CT-RATE cases containing nodule and/or ground-glass-opacity findings. A forward pass was first performed for every case to obtain the classifier's predicted nodule and opacity logits. These logits were also used to rank the cases according to the model's confidence for each target abnormality. However, Grad-CAM heatmaps were generated for all cases in the test set, and the resulting heatmaps were quantitatively evaluated against the corresponding ground-truth lesion masks.

For every test case, four candidate percentile thresholds (70^{th}, 80^{th}, 90^{th}, and 95^{th}) were evaluated. The threshold that produced the highest Dice overlap with the ground-truth lesion mask was selected separately for each case, following the evaluation procedure described in Section **??**. The resulting Dice, IoU, Attribution IoU, pointing-game hit rate, and energy inside the ground-truth mask were then calculated across the full test set. Table 4.5 reports the resulting mean and standard deviation.

Table 4.5: CT-CLIP Grad-CAM localisation metrics evaluated on the full test set (mean ± std).

Metric	**Value**
Dice	0.0019 ± 0.0034
IoU	0.0009 ± 0.0017
Attribution IoU	0.0009 ± 0.0017
Pointing-game hit rate	0.0000 ± 0.0000
Energy inside mask	0.0008 ± 0.0018

Every metric is close to zero, and the pointing-game hit rate is exactly zero. This means that the voxel with the highest heatmap value did not fall inside the annotated lesion in any case in the test set. One important factor contributing to the very low overlap scores is the large difference between the size of the Grad-CAM explanation and the size of the annotated lesion.

The ground-truth annotations cover an average of 59,443 voxels (median 33,099) out of a $480 \times 480 \times 240$ volume containing 55,296,000 voxels, corresponding to only about 0.1% of the full volume. In comparison, even the most selective of the four candidate thresholds (95^{th} percentile) marks 2,764,800 voxels, corresponding to 5% of the volume. Several cases achieved their best Dice score at the 70^{th} or 80^{th} percentile, resulting in 20–30% of the volume being marked as positive. In one case, none of the four candidate thresholds separated the heatmap from the background, resulting in the entire volume being marked as positive.

Therefore, the thresholded Grad-CAM explanation is often much larger than the small lesion it is intended to localise. This large size difference strongly reduces the overlap-based metrics and contributes to the very low Dice and IoU scores. However, the zero pointing-game hit rate also indicates that the strongest individual activation did not reliably fall within the annotated lesion. Thus, the quantitative results suggest that the Grad-CAM heatmaps were not sufficiently precise for localising the small target lesions.

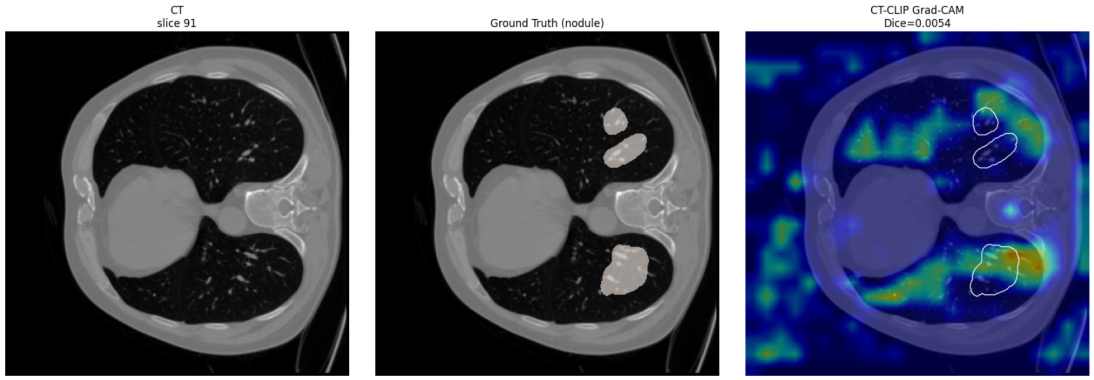

Figure 4.1: Grad-CAM for a nodule case: CT slice (left), ground-truth nodules (centre), and Grad-CAM heatmap with nodule boundaries in white (right).

While the table above summarises the quantitative results for the full test set, Figure 4.1 shows the same behaviour qualitatively for a single case from the test set. This nodule case contains three annotated nodules, with a total of 89,757 ground-truth voxels. At the selected threshold (90^{th} percentile), 5,529,608 voxels were marked as positive. The case achieved a Dice score of 0.0054 and an IoU of 0.0027, which was the highest Dice score among the test-set cases and therefore represents a relatively favourable example rather than a worst-case example.

Even in this case, the strongest heatmap activations were spread broadly across the right lung and chest wall rather than being concentrated around the three annotated nodule boundaries. The single highest-valued voxel also fell outside all three ground-truth annotations, resulting in a pointing-game miss.

This example is consistent with the overall evaluation: the Grad-CAM heatmap generally focuses on the lung region rather than the image background, but it is not precise enough to localise a specific small lesion within the surrounding lung tissue.

4.4 Merlin Results

4.4.1 Zero-Shot Classification Performance

4.4.2 Zero-Shot Localisation Performance

4.5 BiomedParse Results

4.5.1 Presence Detection (Classification) Performance

4.5.2 Segmentation/Localisation Performance

4.6 MedSAM2 Results

4.7 Cross-Model Comparison

4.7.1 Classification Performance Comparison

4.7.2 Localisation/Segmentation Performance Comparison

4.7.3 Explainability Comparison

4.8 Chapter Summary

Chapter 5

Conclusion

Appendix A

Essential English Grammar - A Handout

The Appendices are numbered as A, B,.. and so on. Sections and Subsections of the appendix then would be A.1, A.1.1, A.2 etc.

A.1 Simple Present Tense

A.2 Perfect Tense

A.3 Past Past Perfect Tense

A.4 Passive Voice

A.5 Technical Writing Fundamentals

A.6 Proof Reading

Appendix B

Graphical Interpretation of Data

The graphical interpretation given below shows the relationship between x and y is acceptable to the purpose of the project

B.1 Data Analysis

B.1.1 Statistical Analysis

B.1.2 Empirical Analysis

B.2 The X vs Y vs Z

Bibliography

[1] C. J. Murray, "The global burden of disease study at 30 years," *Nature medicine*, vol. 28, no. 10, pp. 2019–2026, 2022.

[2] K. Doi, "Computer-aided diagnosis in medical imaging: historical review, current status and future potential," *Computerized medical imaging and graphics*, vol. 31, no. 4-5, pp. 198–211, 2007.

[3] A. Borakati, A. Perera, J. Johnson, and T. Sood, "Diagnostic accuracy of x-ray versus ct in covid-19: a propensity-matched database study," *BMJ open*, vol. 10, no. 11, p. e042946, 2020.

[4] G. McCreadie and T. Oliver, "Eight ct lessons that we learned the hard way: an analysis of current patterns of radiological error and discrepancy with particular emphasis on ct," *Clinical radiology*, vol. 64, no. 5, pp. 491–499, 2009.

[5] M. Ennab and H. Mcheick, "Advancing ai interpretability in medical imaging: a comparative analysis of pixel-level interpretability and grad-cam models," *Machine Learning and Knowledge Extraction*, vol. 7, no. 1, p. 12, 2025.

[6] N. L. S. T. R. Team, "The national lung screening trial: overview and study design," *Radiology*, vol. 258, no. 1, pp. 243–253, 2011.

[7] H. J. de Koning, C. M. van Der Aalst, P. A. de Jong, E. T. Scholten, K. Nackaerts, M. A. Heuvelmans, J.-W. J. Lammers, C. Weenink, U. Yousaf-Khan, N. Horeweg, *et al.*, "Reduced lung-cancer mortality with volume ct screening in a randomized trial," *New England journal of medicine*, vol. 382, no. 6, pp. 503–513, 2020.

[8] P. J. Withers, C. Bouman, S. Carmignato, V. Cnudde, D. Grimaldi, C. K. Hagen, E. Maire, M. Manley, A. Du Plessis, and S. R. Stock, "X-ray computed tomography," *Nature Reviews Methods Primers*, vol. 1, no. 1, p. 18, 2021.

[9] K. Doi, "Computer-aided diagnosis in medical imaging: Historical review, current status and future potential," *Computerized Medical Imaging and Graphics*, vol. 31, no. 4, pp. 198–211, 2007. Computer-aided Diagnosis (CAD) and Image-guided Decision Support.

[10] H.-P. Chan, K. Doi, C. Vyborny, K. Lam, and R. Schmidt, "Computer-aided detection of microcalcifications in mammograms. methodology and preliminary clinical study," *Investigative radiology*, vol. 23, pp. 664–71, 10 1988.

[11] T. W. Freer and M. J. Ulissey, "Screening mammography with computer-aided detection: Prospective study of 12,860 patients in a community breast center," *Radiology*, vol. 220, no. 3, pp. 781–786, 2001. PMID: 11526282.

[12] F. Li, M. Aoyama, J. Shiraishi, H. Abe, Q. Li, K. Suzuki, R. Engelmann, S. Sone, H. Macmahon, and K. Doi, "Radiologists' performance for differentiating benign from malignant lung nodules on high-resolution ct using computer-estimated likelihood of malignancy," *AJR. American journal of roentgenology*, vol. 183, pp. 1209–15, 12 2004.

[13] J. Shiraishi, H. Abe, R. Engelmann, M. Aoyama, H. MacMahon, and K. Doi, "Computer-aided diagnosis to distinguish benign from malignant solitary pulmonary nodules on radiographs: Roc analysis of radiologists' performance—initial experience," *Radiology*, vol. 227, no. 2, pp. 469–474, 2003. PMID: 12732700.

[14] K. Doi, "Computer-aided diagnosis in medical imaging: Historical review, current status and future potential," *Computerized Medical Imaging and Graphics*, vol. 31, no. 4, pp. 198–211, 2007. Computer-aided Diagnosis (CAD) and Image-guided Decision Support.

[15] T. Zhao, Y. Gu, J. Yang, N. Usuyama, H. H. Lee, S. Kiblawi, T. Naumann, J. Gao, A. Crabtree, J. Abel, *et al.*, "A foundation model for joint segmentation, detection and recognition of biomedical objects across nine modalities," *Nature methods*, vol. 22, no. 1, pp. 166–176, 2025.

[16] J. Wasserthal, H. C. Breit, M. T. Meyer, *et al.*, "Totalsegmentator: Robust segmentation of 104 anatomic structures in ct images," *Radiology: Artificial Intelligence*, vol. 5, no. 5, p. e230024, 2023.

[17] Z. Yang, Z. Chen, Y. Sun, A. Strittmatter, A. Raj, A. Allababidi, J. S. Rink, and F. G. Zöllner, "seg2med: a segmentation-based medical image generation framework using denoising diffusion probabilistic models," *arXiv e-prints*, pp. arXiv–2504, 2025.

[18] L. Blankemeier, A. Kumar, J. P. Cohen, J. Liu, L. Liu, D. Van Veen, S. J. S. Gardezi, H. Yu, M. Paschali, Z. Chen, J.-B. Delbrouck, E. Reis, R. Holland, C. Truyts, C. Bluethgen, Y. Wu, L. Lian, M. E. K. Jensen, S. Ostmeier, M. Varma, J. M. J. Valanarasu, Z. Fang, Z. Huo, Z. Nabulsi, D. Ardila, W.-H. Weng, E. A. Junior, N. Ahuja, J. Fries, N. H. Shah, G. Zaharchuk, M. Willis, A. Yala, A. Johnston, R. D. Boutin, A. Wentland, C. P. Langlotz, J. Hom, S. Gatidis, and A. S. Chaudhari, "Merlin: a computed tomography vision–language foundation model and dataset," *Nature*, vol. 652, p. 1318–1328, Mar. 2026.

[19] J. Ma, Y. He, F. Li, L. Han, C. You, and B. Wang, "Segment anything in medical images," *Nature Communications*, vol. 15, p. 654, 2024.

[20] N. Ravi, J. Ma, Y. He, B. Wang, *et al.*, "Medsam2: Segment anything in 3d medical images and videos," *arXiv preprint arXiv:2504.03600*, 2025.

[21] E. M. Ayllon *et al.*, "Can foundation models really segment tumors? a benchmarking odyssey in lung ct imaging," *arXiv preprint arXiv:2505.01239*, 2025.

[22] Meta AI, "Sam 3: Segment anything with concepts." Submitted to ICLR 2026 (under review), 2025.

[23] A. Liu, R. Xue, X. R. Cao, Y. Shen, Y. Lu, X. Li, Q. Chen, and J. Chen, "Medsam3: Delving into segment anything with medical concepts," *arXiv preprint arXiv:2511.19046*, 2025.

[24] I. E. Hamamci, S. Er, *et al.*, "Ct-clip: Contrastive vision-language pretraining for chest ct," *arXiv preprint arXiv:2403.17834*, 2024.

[25] I. E. Hamamci, S. Er, E. Almas, E. Keles, E. Keles, M. Basar, A. Gungor, and G. Unal, "Ct-rate: A large-scale dataset for chest ct radiology report generation and abnormality detection," *arXiv preprint arXiv:2401.07515*, 2024.

[26] E. Tjoa and C. Guan, "A survey on explainable artificial intelligence (xai): Toward medical xai," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 11, pp. 4793–4813, 2021.

[27] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?: Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, ACM, 2016.

[28] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.

[29] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.

[30] A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks," in *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pp. 839–847, IEEE, 2018.

[31] H. Wang, Z. Wang, M. Du, F. Yang, Z. Zhang, S. Ding, P. Mardziel, and X. Hu, "Score-cam: Score-weighted visual explanations for convolutional neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 111–119, 2020.

[32] P.-T. Jiang, C.-B. Zhang, Q. Hou, M.-M. Cheng, and Y. Wei, "Layercam: Exploring hierarchical class activation maps for localization," *IEEE transactions on image processing*, vol. 30, pp. 5875–5888, 2021.

[33] V. Hassija, V. Chamola, A. Mahapatra, A. Singal, D. Goel, K. Huang, S. Scardapane, I. Spinelli, M. Mahmud, and A. Hussain, "Interpreting black-box models: a review on explainable artificial intelligence," *Cognitive Computation*, vol. 16, no. 1, pp. 45–74, 2024.

[34] L. Ribeiro-Flucht, X. Chen, and D. Meurers, "Explainable ai in language learning: Linking empirical evidence and theoretical concepts in proficiency and readability modeling of portuguese," in *Proceedings of the 19th workshop on innovative use of NLP for building educational applications (BEA 2024)*, pp. 199–209, 2024.

[35] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.

[36] R. Guidotti, "Counterfactual explanations and how to find them: literature review and benchmarking: R. guidotti," *Data Mining and Knowledge Discovery*, vol. 38, no. 5, pp. 2770–2824, 2024.

[37] M. Van Lent, W. Fisher, and M. Mancuso, "An explainable artificial intelligence system for small-unit tactical behavior," in *Proceedings of the national conference on artificial intelligence*, pp. 900–907, Menlo Park, CA; Cambridge, MA; London; AAAI Press; MIT Press; 1999, 2004.

[38] H. Zheng, H. Yong, and L. Zhang, "Deep convolutional dictionary learning for image denoising," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 630–641, 2021.

[39] Y. Li, S. Ding, and Z. Li, "Dictionary learning with the cosparse analysis model based on summation of blocked determinants as the sparseness measure," *Digital Signal Processing*, vol. 48, pp. 298–309, 2016.

[40] H. Lee, A. Battle, R. Raina, and A. Ng, "Efficient sparse coding algorithms," in *Advances in Neural Information Processing Systems* (B. Schölkopf, J. Platt, and T. Hoffman, eds.), vol. 19, MIT Press, 2006.

[41] M. Aharon, M. Elad, and A. Bruckstein, "K-svd: An algorithm for designing overcomplete dictionaries for sparse representation," *IEEE Transactions on Signal Processing*, vol. 54, no. 11, pp. 4311–4322, 2006.

[42] A. Yahyaoui, N. Fnaiech, and F. Fnaiech, "A suitable initialization procedure for speeding a neural network job-shop scheduling," *IEEE Transactions on Industrial Electronics*, vol. 58, no. 3, pp. 1052–1060, 2011.

[43] Y. Pati, R. Rezaiifar, and P. Krishnaprasad, "Orthogonal matching pursuit: recursive function approximation with applications to wavelet decomposition," in *Proceedings of 27th Asilomar Conference on Signals, Systems and Computers*, pp. 40–44 vol.1, 1993.

[44] K. Engan, S. Aase, and J. Hakon Husoy, "Method of optimal directions for frame design," in *1999 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings. ICASSP99 (Cat. No.99CH36258)*, vol. 5, pp. 2443–2446 vol.5, 1999.

[45] J. Mairal, F. Bach, J. Ponce, and G. Sapiro, "Online dictionary learning for sparse coding," in *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09, (New York, NY, USA), p. 689–696, Association for Computing Machinery, 2009.

[46] M. Aharon, M. Elad, and A. Bruckstein, "K-svd: An algorithm for designing overcomplete dictionaries for sparse representation," *IEEE Transactions on Signal Processing*, vol. 54, no. 11, pp. 4311–4322, 2006.

[47] Z. Jiang, Z. Lin, and L. S. Davis, "Label consistent k-svd: Learning a discriminative dictionary for recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 11, pp. 2651–2664, 2013.

[48] M. Yang, L. Zhang, X. Feng, and D. Zhang, "Fisher discrimination dictionary learning for sparse representation," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 543–550, 2011.

[49] B. T. Carroll, B. M. Whitaker, W. Dayley, and D. V. Anderson, "Outlier learning via augmented frozen dictionaries," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 25, no. 6, pp. 1207–1219, 2017.

[50] T. Tong, J. Caballero, K. Bhatia, and D. Rueckert, "Dictionary learning for medical image denoising, reconstruction, and segmentation," in *Machine learning and medical imaging*, pp. 153–181, Elsevier, 2016.

[51] M. Elad and M. Aharon, "Image denoising via sparse and redundant representations over learned dictionaries," *IEEE Transactions on Image processing*, vol. 15, no. 12, pp. 3736–3745, 2006.

[52] L. Bao, M. Robini, W. Liu, and Y. Zhu, "Structure-adaptive sparse denoising for diffusion-tensor mri," *Medical image analysis*, vol. 17, no. 4, pp. 442–457, 2013.

[53] L. Shi, Y. Chen, H. Shu, L. Luo, C. Toumoulin, and J.-L. Coatrieux, "Low-dose ct image processing using artifact suppressed dictionary learning," in *2014 IEEE 11th International Symposium on Biomedical Imaging (ISBI)*, pp. 1127–1130, IEEE, 2014.

[54] S. Ravishankar and Y. Bresler, "Mr image reconstruction from highly undersampled k-space data by dictionary learning," *IEEE Transactions on Medical Imaging*, vol. 30, no. 5, pp. 1028–1041, 2011.

[55] A. Rueda, N. Malpica, and E. Romero, "Single-image super-resolution of brain mr images using overcomplete dictionaries," *Medical image analysis*, vol. 17, no. 1, pp. 113–132, 2013.

[56] M. Liu, L. Lu, X. Ye, S. Yu, and M. Salganicoff, "Sparse classification for computer aided diagnosis using learned dictionaries," in *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pp. 41–48, Springer, 2011.

[57] P. Wu, K. Xia, Y. Zhang, X. Qian, G. Wang, and H. Yu, "Dictionary learning-based ct detection of pulmonary nodules," in *Developments in X-Ray Tomography X*, vol. 9967, pp. 88–99, SPIE, 2016.

[58] S. Kong and D. Wang, "A dictionary learning approach for classification: Separating the particularity and the commonality," in *European conference on computer vision*, pp. 186–199, Springer, 2012.

[59] A. Ben-Cohen, E. Klang, A. Kerpel, E. Konen, M. M. Amitai, and H. Greenspan, "Fully convolutional network and sparsity-based dictionary learning for liver lesion detection in ct examinations," *Neurocomputing*, vol. 275, pp. 1585–1594, 2018.

[60] X. Zhang, C. Wu, Z. Zhao, J. Lei, W. Tian, Y. Zhang, W. Xie, and Y. Wang, "Development of a large-scale grounded vision language dataset for chest ct analysis," *Scientific Data*, vol. 12, no. 1, p. 1636, 2025.

[61] R. L. Draelos, D. Dov, M. A. Mazurowski, J. Y. Lo, R. Henao, G. D. Rubin, and L. Carin, "Machine-learning-based multiple abnormality prediction with large-scale chest computed tomography volumes," *Medical image analysis*, vol. 67, p. 101857, 2021.

[62] P. Rajpurkar *et al.*, "Rexgroundingct: A large-scale dataset for grounded radiology report generation on chest ct," *arXiv preprint*, 2024.

[63] X. Zhang, H.-Y. Zhou, X. Yang, O. Banerjee, J. N. Acosta, J. Miller, O. Huang, and P. Rajpurkar, "Rexrank: A public leaderboard for ai-powered radiology report generation," 2024.

[64] J.-x. Yang, M. Zhang, Z.-h. Liu, L. Ba, J.-x. Gan, and S.-w. Xu, "Detection of lung atelectasis/consolidation by ultrasound in multiple trauma patients with mechanical ventilation," *Critical Ultrasound Journal*, vol. 1, no. 1, pp. 13–16, 2009.

[65] Z. Zhao, L. Dai, Y. Zhang, W. Xie, and Y. Wang, "Rethinking whole-body ct image interpretation: An abnormality-centric approach," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5179–5189, 2026.

[66] L. Welch, "Lower bounds on the maximum cross correlation of signals (corresp.)," *IEEE Transactions on Information theory*, vol. 20, no. 3, pp. 397–399, 1974.

[67] J. Mairal, F. Bach, J. Ponce, and G. Sapiro, "Online learning for matrix factorization and sparse coding," *Journal of Machine Learning Research*, vol. 11, pp. 19–60, 2010.

[68] A. Beck and M. Teboulle, "A fast iterative shrinkage-thresholding algorithm for linear inverse problems," *SIAM Journal on Imaging Sciences*, vol. 2, no. 1, pp. 183–202, 2009.